1. Исследование объявлений о продаже квартир

Для данного проекта имеются данные сервиса Яндекс Недвижимость — архив объявлений о продаже квартир в Санкт-Петербурге и соседних населённых пунктах за несколько лет. Нужно научиться определять рыночную стоимость объектов недвижимости. Задача — установить параметры. Это позволит построить автоматизированную систему: она отследит аномалии и мошенническую деятельность. По каждой квартире на продажу доступны два вида данных. Первые вписаны пользователем, вторые — получены автоматически на основе картографических данных. Например, расстояние до центра, аэропорта, ближайшего парка и водоёма.

Имеющиеся признаки:

- airports_nearest — расстояние до ближайшего аэропорта в метрах (м)
- balcony — число балконов
- ceiling_height — высота потолков (м)
- cityCenters_nearest — расстояние до центра города (м)
- days_exposition — сколько дней было размещено объявление (от публикации до снятия)
- first_day_exposition — дата публикации
- floor — этаж
- floors_total — всего этажей в доме
- is_apartment — апартаменты (булев тип)
- kitchen_area — площадь кухни в квадратных метрах ($м^2$)
- last_price — цена на момент снятия с публикации
- living_area — жилая площадь в квадратных метрах ($м^2$)
- locality_name — название населённого пункта
- open_plan — свободная планировка (булев тип)
- parks_around3000 — число парков в радиусе 3 км
- parks_nearest — расстояние до ближайшего парка (м)
- ponds_around3000 — число водоёмов в радиусе 3 км
- ponds_nearest — расстояние до ближайшего водоёма (м)
- rooms — число комнат
- studio — квартира-студия (булев тип)
- total_area — площадь квартиры в квадратных метрах ($м^2$)
- total_images — число фотографий квартиры в объявлении

1.1. Изучение данных из файла

1.1.1. Создаем базу знаний о столбцах

Сначала выделим то, что знаем - иногда может пригодиться быстрый доступ к пояснению названия столбца

Ввод [57]:

```
s = '''airports_nearest — расстояние до ближайшего аэропорта в метрах (м)
balcony — число балконов
ceiling_height — высота потолков (м)
cityCenters_nearest — расстояние до центра города (м)
days_exposition — сколько дней было размещено объявление (от публикации до снятия)
first_day_exposition — дата публикации
floor — этаж
floors_total — всего этажей в доме
is_apartment — апартаменты (булев тип)
kitchen_area — площадь кухни в квадратных метрах (м²)
last_price — цена на момент снятия с публикации
living_area — жилая площадь в квадратных метрах (м²)
locality_name — название населённого пункта
open_plan — свободная планировка (булев тип)
parks_around3000 — число парков в радиусе 3 км
parks_nearest — расстояние до ближайшего парка (м)
ponds_around3000 — число водоёмов в радиусе 3 км
ponds_nearest — расстояние до ближайшего водоёма (м)
rooms — число комнат
studio — квартира-студия (булев тип)
total_area — площадь квартиры в квадратных метрах (м²)
total_images — число фотографий квартиры в объявлении'''

knowledge = {}

for i in s.split('\n'):
    knowledge[i.split(' — ')[0]] = i.split(' — ')[1]
```

1.1.2. Исследуем данные из файла

Ввод [250]:

```
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

Разделитель не по запятой, а по \t

Ввод [251]:

```
data = pd.read_csv('/datasets/real_estate_data.csv', sep='\t')
```

Посмотрим на данные

Ввод [252]:

```
data.head()
```

Out[252]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07T00:00:00	3	2.70	16.0
1	7	3350000.0	40.4	2018-12-04T00:00:00	1	NaN	11.0
2	10	5196000.0	56.0	2015-08-20T00:00:00	2	NaN	5.0
3	0	64900000.0	159.0	2015-07-24T00:00:00	3	NaN	14.0
4	2	10000000.0	100.0	2018-06-19T00:00:00	2	3.03	14.0

5 rows × 22 columns

Ввод [253]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23699 entries, 0 to 23698
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   total_images          23699 non-null  int64  
 1   last_price            23699 non-null  float64
 2   total_area            23699 non-null  float64
 3   first_day_exposition  23699 non-null  object 
 4   rooms                 23699 non-null  int64  
 5   ceiling_height        14504 non-null  float64
 6   floors_total          23613 non-null  float64
 7   living_area           21796 non-null  float64
 8   floor                 23699 non-null  int64  
 9   is_apartment          2775 non-null   object 
 10  studio                23699 non-null  bool   
 11  open_plan             23699 non-null  bool   
 12  kitchen_area          21421 non-null  float64
 13  balcony               12180 non-null  float64
 14  locality_name         23650 non-null  object 
 15  airports_nearest      18157 non-null  float64
 16  cityCenters_nearest   18180 non-null  float64
 17  parks_around3000      18181 non-null  float64
 18  parks_nearest         8079 non-null   float64
 19  ponds_around3000      18181 non-null  float64
 20  ponds_nearest         9110 non-null   float64
 21  days_exposition       20518 non-null  float64
dtypes: bool(2), float64(14), int64(3), object(3)
memory usage: 3.7+ MB
```

Выведем только те строки, в которых есть пропущенные значения, их тип и

подсчитаем количество пропусков

Ввод [254]:

```
for column in data.columns:
    if any(data[column].isna()):
        print(column, '-', knowledge[column])
        print(data[column].dtype)
        print(data[column].isna().sum(), ' | ', round(data[column].isna().sum()/2369
        print()
```

```
ceiling_height - высота потолков (м)
float64
9195  |  38.8 %

floors_total - всего этажей в доме
float64
86  |  0.4 %

living_area - жилая площадь в квадратных метрах (м²)
float64
1903  |  8.0 %

is_apartment - апартаменты (булев тип)
object
20924  |  88.3 %

kitchen_area - площадь кухни в квадратных метрах (м²)
float64
2278  |  9.6 %

balcony - число балконов
float64
11519  |  48.6 %

locality_name - название населённого пункта
object
49  |  0.2 %

airports_nearest - расстояние до ближайшего аэропорта в метрах (м)
float64
5542  |  23.4 %

cityCenters_nearest - расстояние до центра города (м)
float64
5519  |  23.3 %

parks_around3000 - число парков в радиусе 3 км
float64
5518  |  23.3 %

parks_nearest - расстояние до ближайшего парка (м)
float64
15620  |  65.9 %

ponds_around3000 - число водоёмов в радиусе 3 км
float64
5518  |  23.3 %

ponds_nearest - расстояние до ближайшего водоёма (м)
float64
14589  |  61.6 %
```

```
days_exposition - сколько дней было размещено объявление (от публикаци
и до снятия)
float64
3181  |  13.4 %
```

1.1.3. Вывод

В основном у нас пропущены числовые значения, с которыми работать проще

Так же есть пропуски в апартаментах, будет необходимо выяснить закономерности, в балконах (там проще, балконов скорее всего нет) и в расстоянии до ближайших парков и водоемов

Остальные пропуски значительно меньше

1.2. Предобработка данных

1.2.1. Количество этажей в доме

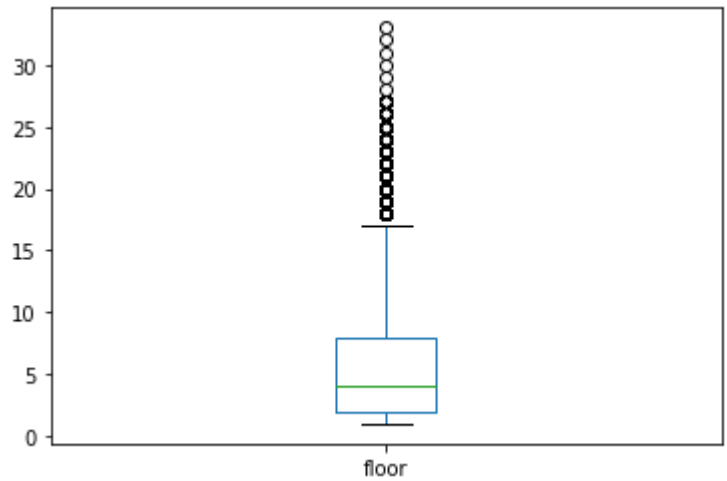
Посмотрим на боксплот наших данных

Ввод [255]:

```
data['floor'].plot(kind='box')
```

Out[255]:

```
<AxesSubplot:>
```



В целом, выглядит логично - медиана между 3 и 5, 75% данных не превышают ~8 этажей. Остальные видятся как выбросы, но и высотками Санкт-Петербург не обделен

Есть предположение, что число этажей не указывают тогда, когда это частный дом, стоит проверить

Ввод [256]:

```
data.query('floors_total.isna()')[['floors_total', 'floor']].head()
```

Out[256]:

	floors_total	floor
186	NaN	4
237	NaN	1
457	NaN	12
671	NaN	8
1757	NaN	9

Неверно, этажи там есть

Можно заполнить медианными значениями, но главное - учитывать этаж при заполнении

Нам такая конструкция пригодится еще не раз, создадим универсальную

Ввод [257]:

```
# запоминаем все данные, которые оказались пропущены
lost_data = data.query('floors_total.isna()')['floor'].unique()
# создаем словарь для хранения показателей известных данных
lost_data_median = {}
# пробегаемся по элементам пропущенных данных
for item in lost_data:
    # заполняем словарик
    lost_data_median[item] = (data
                                  # просим все такие же данные с известными показател
                                  .query('floor == @item and not floors_total.isna()'
                                  # берем медианное значение
                                  .median()
                                 )
# заполняем все пропущенные значения в соответствии со словарем
for row in data.query('floors_total.isna()').index:
    data.loc[row, 'floors_total'] = lost_data_median[data.loc[row]['floor']]
```

Ввод [258]:

```
# повторяем упражнение
data.query('floors_total.isna()')[['floors_total', 'floor']]
```

Out[258]:

	floors_total	floor

1.2.2. Высота потолков

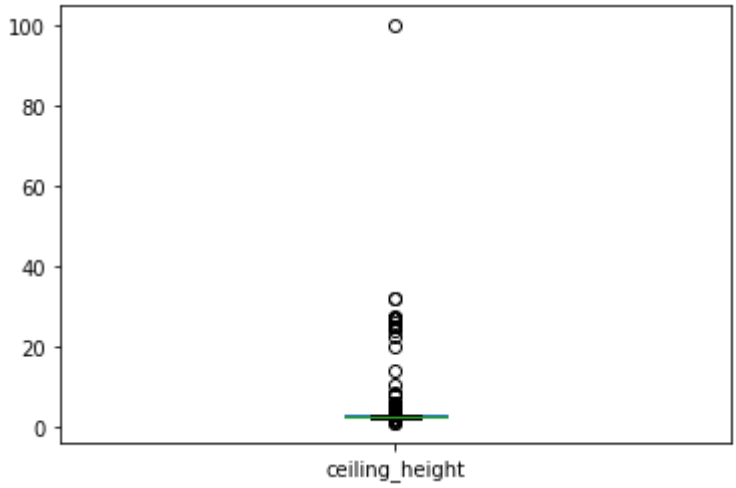
Посмотрим на ошибки в высоте потолков

Ввод [260]:

```
data['ceiling_height'].plot(kind='box')
```

Out[260]:

```
<AxesSubplot:>
```



Сразу бросается в глаза выброс под 100м высоты потолка - надо разобраться, куда им столько

Ввод [261]:

```
(data
 .query('ceiling_height > 5')
 .sort_values(by='ceiling_height', ascending=False)
 .head(8))
```

Out[261]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_to
22869	0	15000000.0	25.0	2018-07-25T00:00:00	1	100.0	5
22336	19	9999000.0	92.4	2019-04-05T00:00:00	2	32.0	6
3148	14	2900000.0	75.0	2018-11-12T00:00:00	3	32.0	3
21377	19	4900000.0	42.0	2017-04-18T00:00:00	1	27.5	24
22938	14	4000000.0	98.0	2018-03-15T00:00:00	4	27.0	2
10773	8	3800000.0	58.0	2017-10-13T00:00:00	2	27.0	10
21824	20	2450000.0	44.0	2019-02-12T00:00:00	2	27.0	2
4876	7	3000000.0	25.0	2017-09-27T00:00:00	0	27.0	25

8 rows × 22 columns

В помещении общей площадью 25 кв. м высота потолка даже 10м уже будет странно, так что скорее всего это очень странный выброс

Ввод [263]:

```
data = data[(data['ceiling_height'] < 100) | (data['ceiling_height'].isna())]
```

Все остальные показатели можно предположить как указанные в дециметрах (веселая система, конечно), то есть 20м это на самом деле 2м, и все что больше 20 можно заменить на десятую часть

Ввод [160]:

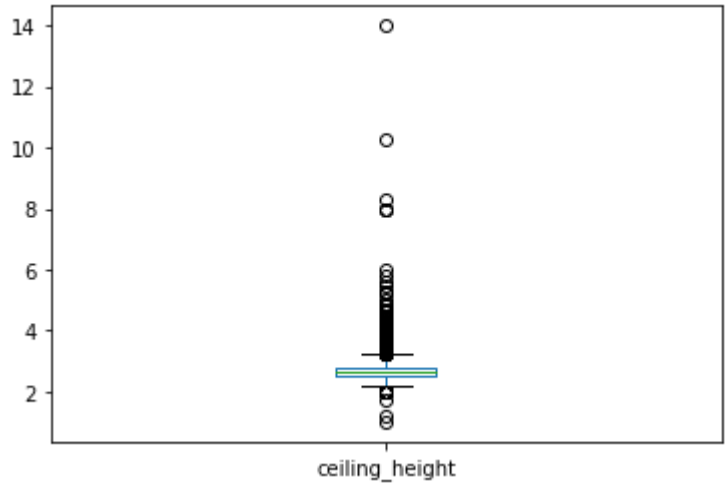
```
for i in data.query('ceiling_height >= 20').index:
    data.loc[i, 'ceiling_height'] = data.loc[i, 'ceiling_height']/10
```

Ввод [161]:

```
data['ceiling_height'].plot(kind='box')
```

Out[161]:

```
<AxesSubplot:>
```



Лично я готов предположить, что в особенных домах вполне возможны потолки под 14м, так что эти данные оставляем

Посмотрим, сможем ли мы опираться на количество этажей в подобных квартирах

Ввод [162]:

```
data.query('ceiling_height.isna()')[['ceiling_height', 'floors_total']].head()
```

Out[162]:

ceiling_height	floors_total

Думаю, сможем, и можно поступить так же как и в прошлый раз

Ввод [163]:

```
lost_ceiling = list(data.query('ceiling_height.isna()')['floors_total'].unique())
lost_ceiling_median = {}
for floor in lost_ceiling:
    lost_ceiling_median[floor] = (data
                                  .query('floor == @floor and not ceiling_height.isna
                                  .median())
for row in data.query('ceiling_height.isna()').index:
    data.loc[row, 'ceiling_height'] = lost_ceiling_median[data.loc[row]['floors_tota
```

Ввод [164]:

```
data.query('ceiling_height.isna()')[['ceiling_height', 'floors_total', 'floor']].hea
```

Out[164]:

ceiling_height	floors_total	floor

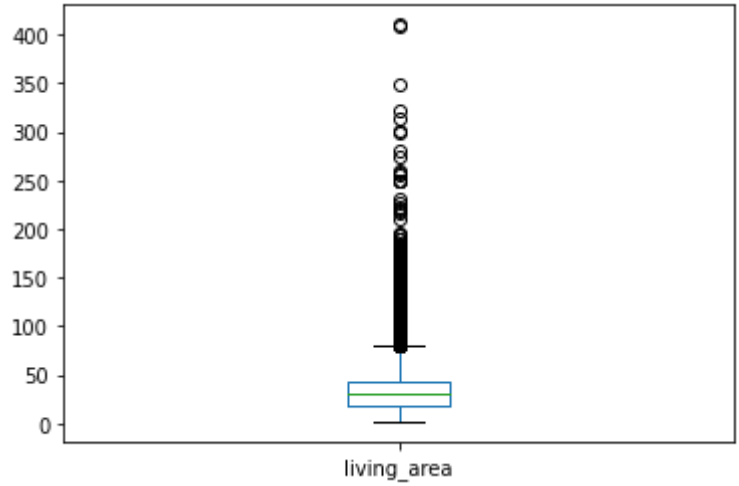
1.2.3. Жилая площадь

Ввод [165]:

```
data['living_area'].plot(kind='box')
```

Out[165]:

```
<AxesSubplot:>
```



В целом, никаких аномальных выбросов не видно

Посмотрим, какие у нас есть пропуски

Ввод [166]:

```
data.query('living_area.isna()').head()
```

Out[166]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
37	10	1990000.0	45.8	2017-10-28T00:00:00	2	2.50	5.0
116	18	10999000.0	97.2	2017-11-13T00:00:00	3	2.50	16.0
165	17	7430000.0	60.2	2019-02-28T00:00:00	2	2.50	4.0
227	13	2200000.0	47.0	2018-01-12T00:00:00	3	2.34	5.0
276	5	23900000.0	108.4	2018-11-15T00:00:00	3	3.06	7.0

5 rows × 22 columns

Посмотрим, сколько раз пропускались и жилая и общая площадь

Ввод [167]:

```
data.query('living_area.isna() and total_area.isna()')
```

Out[167]:

total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total	livi

0 rows × 22 columns

0 раз - это хорошо, значит, мы сможем опираться на их медианное значение

Но так просто тут уже не выйдет - площади уникальны, и ориентироваться надо с разбросом +/- 5 кв. м

Ввод [168]:

```
lost_area = [int(i) for i in data.query('living_area.isna()')['total_area'].unique()
lost_area_median = {}
for total in lost_area:
    lost_area_median[total] = (data
                               .query('@total-5 < total_area < @total+5 and not li
                               .median())
for row in data.query('living_area.isna()').index:
    data.loc[row, 'living_area'] = lost_area_median[int(data.loc[row]['total_area'])
```

Ввод [169]:

```
data.query('living_area.isna()')
```

Out[169]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_t
15016	8	150000000.0	500.0	2016-01-13T00:00:00	4	3.20	
19392	5	71000000.0	371.0	2018-07-26T00:00:00	0	3.57	
22131	14	27000000.0	422.2	2018-10-18T00:00:00	5	2.70	

3 rows × 22 columns

Остались 3 самых непоколебимых квартиры. Значения общей площади очень большие, посмотрим, сколько подобных квартир есть в районе +/- 50 кв. м

Ввод [170]:

```
print('370:', data.query('390 < total_area < 590 and not living_area.isna()').shape)
print('500:', data.query('450 < total_area < 550 and not living_area.isna()').shape)
print('420:', data.query('370 < total_area < 570 and not living_area.isna()').shape)
```

```
370: (14, 22)
500: (7, 22)
420: (15, 22)
```

Очень уважаемо, нам хватит на средние. Применим к ним отдельно

Ввод [171]:

```
lost_area = [int(i) for i in data.query('living_area.isna()')['total_area'].unique()
lost_area_median = {}
for total in lost_area:
    lost_area_median[total] = (data
                                .query('@total-50 < total_area < @total+50 and not
                                .median()
                               )
for row in data.query('living_area.isna()').index:
    data.loc[row, 'living_area'] = lost_area_median[int(data.loc[row]['total_area'])
```

Ввод [172]:

```
data.query('living_area.isna()')
```

Out[172]:

total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total	livi

0 rows × 22 columns

Прекрасно, с ними мы тоже справились

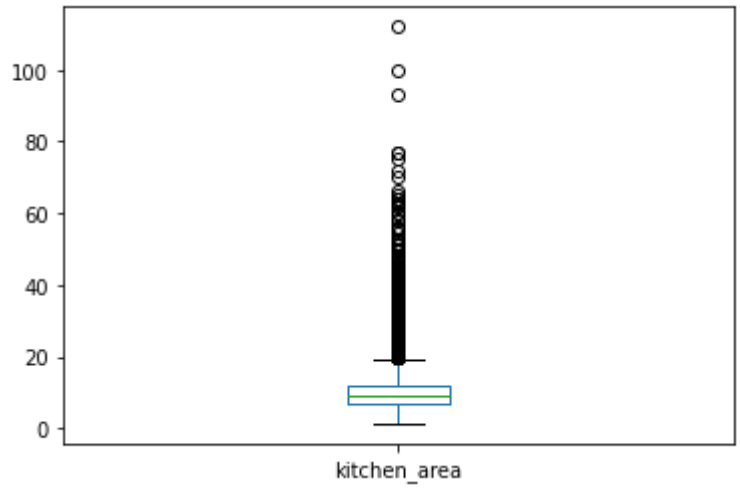
1.2.4. Площадь кухни

Ввод [173]:

```
data['kitchen_area'].plot(kind='box')
```

Out[173]:

```
<AxesSubplot:>
```



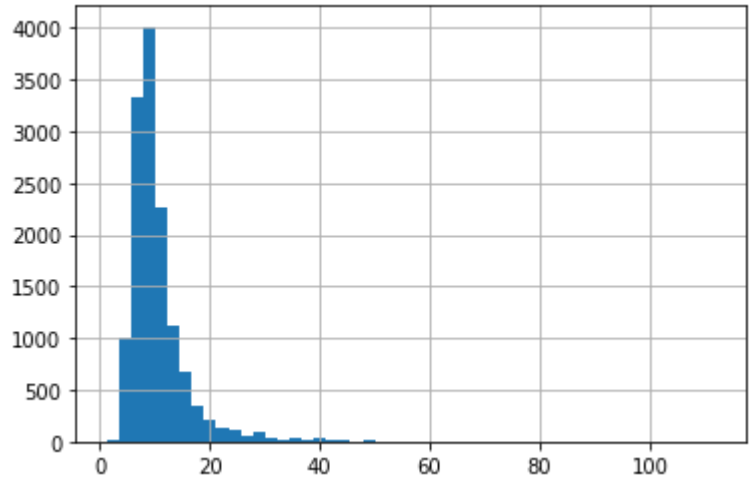
Очень странные данные, кухни за 100 кв. м?

Ввод [174]:

```
data['kitchen_area'].hist(bins=50)
```

Out[174]:

```
<AxesSubplot:>
```



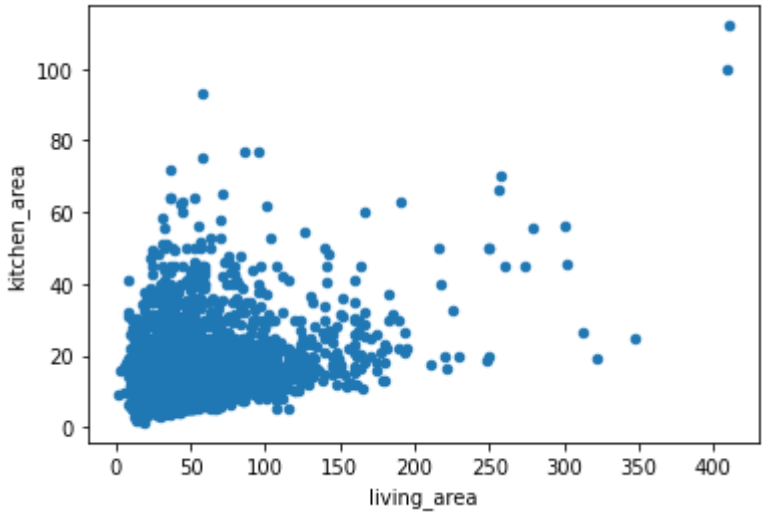
Не так много, но выше 40 кв. м кухни все таки есть

Ввод [175]:

```
data.sort_values(by='kitchen_area').plot(x='living_area', y='kitchen_area', kind='sc
```

Out[175]:

```
<AxesSubplot:xlabel='living_area', ylabel='kitchen_area'>
```



И, как выяснилось, есть даже те, у которых своя площадь больше, чем площадь квартиры

Ввод [176]:

```
data.query('kitchen_area >= living_area')[['living_area', 'kitchen_area']]
```

Out[176]:

	living_area	kitchen_area
4	32.00	41.00
51	38.00	50.00
101	12.00	13.00
263	39.80	49.20
379	20.00	36.00
...	...	...
22597	16.40	17.50
23148	20.00	28.00
23573	13.59	17.87
23574	3.00	16.00
23658	14.00	15.50

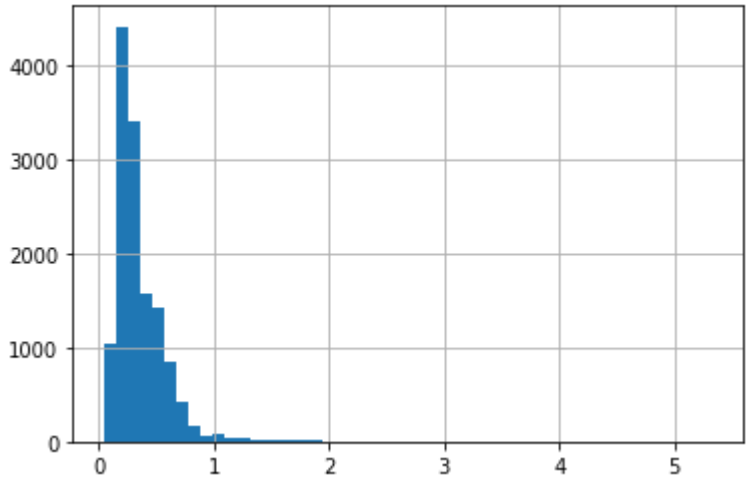
236 rows × 2 columns

Ввод [177]:

```
(data['kitchen_area'] / data['living_area']).hist(bins=50)
```

Out[177]:

```
<AxesSubplot:>
```



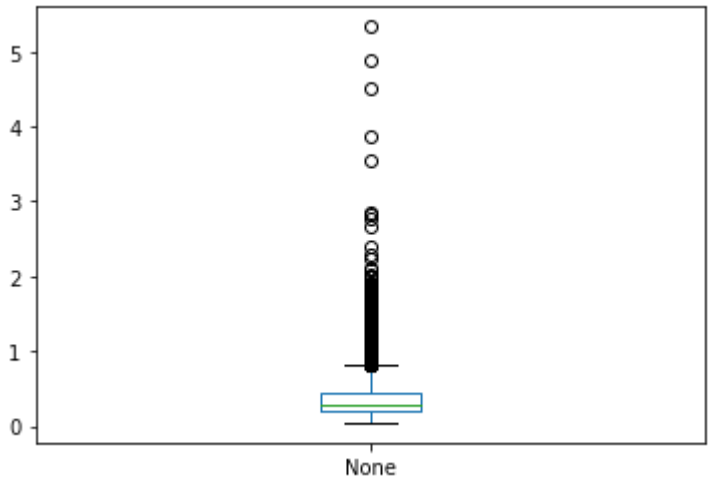
Соотношения не выглядят очень радостно: большинство столпилось около 20-30% от помещения, но есть и больше 100%

Ввод [178]:

```
(data['kitchen_area'] / data['living_area']).plot(kind='box')
```

Out[178]:

```
<AxesSubplot:>
```



Все, что выше единицы – применение магии вне Хогвартса, а нас такое не интересует

Ввод [179]:

```
data.query('kitchen_area.isna()').head()
```

Out[179]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
37	10	1990000.0	45.8	2017-10-28T00:00:00	2	2.50	5.0
155	8	96000000.0	317.0	2017-07-07T00:00:00	4	3.50	3.0
178	18	16300000.0	153.0	2019-03-30T00:00:00	4	2.71	10.0
188	15	4600000.0	38.5	2019-04-16T00:00:00	1	2.50	12.0
199	4	3735325.0	37.4	2019-03-22T00:00:00	1	2.73	12.0

5 rows × 22 columns

Чтобы брать адекватные медианные значения - надо мыслить как адекватные медианные значения, а знать выкинуть все кухни, площадь которых больше 80% квартиры (знавал я квартиры, где кухня реально 60-70% занимает)

Как и с жилой площадью, будем ориентироваться на известные показатели - как раз на жилую площадь

Ввод [180]:

```
lost_area = [int(i) for i in data.query('kitchen_area.isna()')['living_area'].unique
lost_area_median = {}
for total in lost_area:
    lost_area_median[total] = (data
                               .query('@total-5 < living_area < @total+5')
                               # доп. условие на соотношение площади кухни
                               .query('kitchen_area < 0.8 * living_area')
                               .query('not kitchen_area.isna()')['kitchen_area']
                               .median()
                               )
for row in data.query('kitchen_area.isna()').index:
    data.loc[row, 'kitchen_area'] = lost_area_median[int(data.loc[row]['living_area'
```

Ввод [181]:

```
data.query('kitchen_area.isna()').head()
```

Out[181]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total	livi

0 rows × 22 columns

Теперь разберемся с оставшимися кухнями, больше площади квартиры

Можно так же предположить, что случайно не дописали запятую, и на самом деле у нас кухня при 30 кв.м жилья не 41 кв.м, а всего 4 кв.м, что выглядит намного более логично

Ввод [182]:

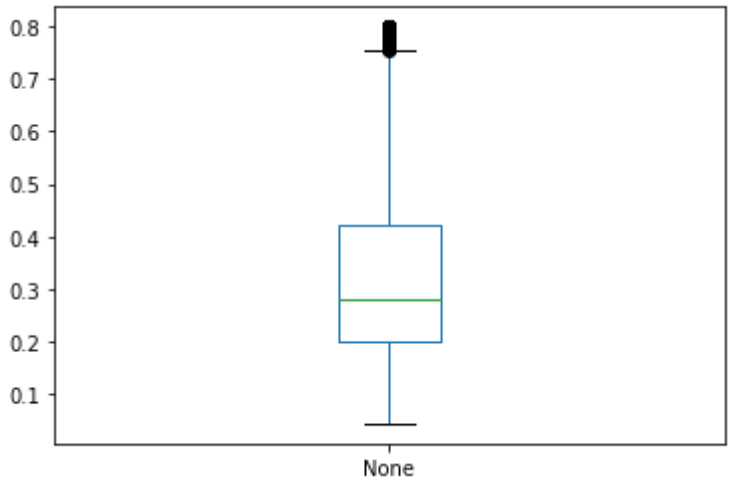
```
for row in data.query('kitchen_area > 0.8 * living_area').index:
    data.loc[row, 'kitchen_area'] = data.loc[row, 'kitchen_area'] / 10
```

Ввод [183]:

```
(data['kitchen_area'] / data['living_area']).plot(kind='box')
```

Out[183]:

```
<AxesSubplot:>
```



Намного больше похоже на правду

1.2.5. Балконы

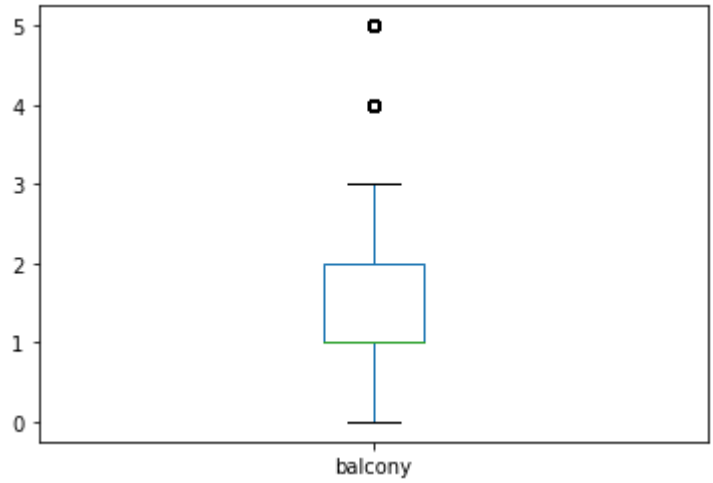
Если нет информации о балконах - скорее всего, нет балконов

Ввод [184]:

```
data['balcony'].plot(kind='box')
```

Out[184]:

```
<AxesSubplot:>
```



Ввод [185]:

```
# проставляем в пропусках балконов нули
data['balcony'] = data['balcony'].fillna(0)
```

Ввод [186]:

```
data.query('balcony.isna()')
```

Out[186]:

total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total	livi

0 rows × 22 columns

Замечательно

1.2.6. Название населённого пункта

49 пропусков - поставим в "Остальные"

Ввод [187]:

```
data.query('locality_name.isna()').head()
```

Out[187]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_to
1097	3	8600000.0	81.7	2016-04-15T00:00:00	3	3.55	5
4151	17	17600000.0	89.5	2014-12-09T00:00:00	2	3.00	8
4189	7	9200000.0	80.0	2015-12-10T00:00:00	3	4.00	4
9821	13	8000000.0	94.5	2015-01-21T00:00:00	4	3.00	2
13223	1	2919911.0	29.4	2015-03-12T00:00:00	1	2.75	24

5 rows × 22 columns

Ввод [188]:

```
data['locality_name'] = data['locality_name'].fillna('Остальные')
```

Ввод [189]:

```
data.query('locality_name.isna()')
```

Out[189]:

total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total	livi

0 rows × 22 columns

1.2.7. Срок размещения объявления

Как раз самое время переделать столбцы со временем в красивый тип данных

Ввод [190]:

```
data['first_day_exposition']= pd.to_datetime(data['first_day_exposition'], format='%
```

Ввод [191]:

```
data.head()
```

Out[191]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07	3	2.70	16.0
4	2	10000000.0	100.0	2018-06-19	2	3.03	14.0
9	18	5400000.0	61.0	2017-02-26	3	2.50	9.0
10	5	5050000.0	39.6	2017-11-16	1	2.67	12.0
13	20	3550000.0	42.8	2017-07-01	2	2.56	5.0

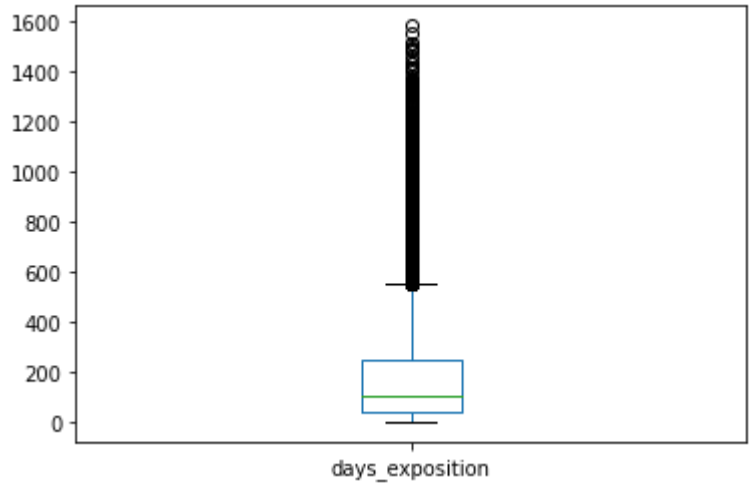
5 rows × 22 columns

Ввод [192]:

```
data['days_exposition'].plot(kind='box')
```

Out[192]:

```
<AxesSubplot:>
```



Пропуски в дате снятия объявления появляются из за того, что объявление еще активно

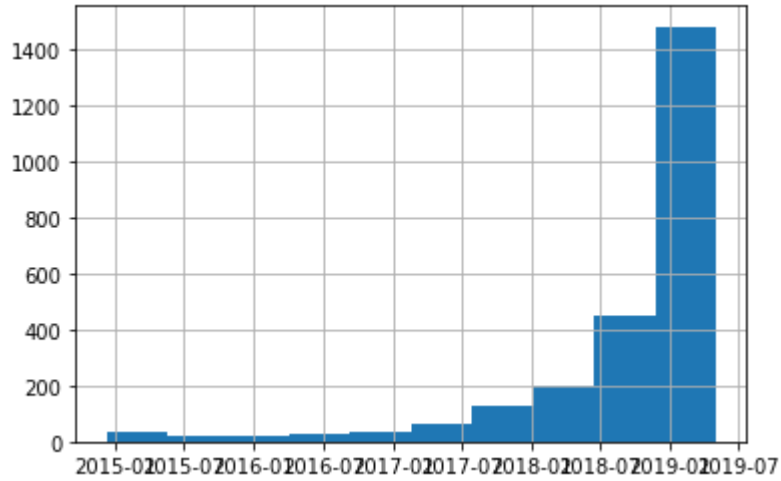
Надо проверить

Ввод [193]:

```
data.query('days_exposition.isna()')['first_day_exposition'].hist()
```

Out[193]:

```
<AxesSubplot:>
```



Не зная, сколько это объявление еще провисит на сайте - не будем подставлять сегодняшний день как финальный

1.2.8. Остальное

Посмотрим, что у нас осталось с пропусками

Ввод [194]:

```
for column in data.columns:
    if any(data[column].isna()):
        print(column, '-', knowledge[column])
        print(data[column].dtype)
        print(data[column].isna().sum(), ' | ', round(data[column].isna().sum()/2369
        print()
```

```
is_apartment - апартаменты (булев тип)
object
12642  |  53.3 %

airports_nearest - расстояние до ближайшего аэропорта в метрах (м)
float64
2964  |  12.5 %

cityCenters_nearest - расстояние до центра города (м)
float64
2944  |  12.4 %

parks_around3000 - число парков в радиусе 3 км
float64
2943  |  12.4 %

parks_nearest - расстояние до ближайшего парка (м)
float64
9210  |  38.9 %

ponds_around3000 - число водоёмов в радиусе 3 км
float64
2943  |  12.4 %

ponds_nearest - расстояние до ближайшего водоёма (м)
float64
8772  |  37.0 %

days_exposition - сколько дней было размещено объявление (от публикаци
и до снятия)
float64
2453  |  10.4 %
```

Числа и расстояния рассчитать не представляется возможным – брать среднее нелогично, а высчитывать по похожим – мешает окружность

Для их анализа выглядит разумным заполнить нулями

Ввод [195]:

```
data['parks_around3000'] = data['parks_around3000'].fillna(0)
data['ponds_around3000'] = data['ponds_around3000'].fillna(0)
```

Апартамены можно было бы привязать к площади квартиры, высоте потолков и остальным более-менее причастным данным, не будь этих пропусков 88%, слишком высока погрешность данных

Судя по всему, именно эти данные стоит оставить нетронутыми – люди не указывали их, и вряд ли выйдет верно предположить, как именно обстоят там дела

1.2.9. Изменение типов данных

Ввод [196]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 14503 entries, 0 to 23698
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   total_images          14503 non-null  int64
 1   last_price            14503 non-null  float64
 2   total_area            14503 non-null  float64
 3   first_day_exposition  14503 non-null  datetime64[ns]
 4   rooms                 14503 non-null  int64
 5   ceiling_height        14503 non-null  float64
 6   floors_total          14503 non-null  float64
 7   living_area           14503 non-null  float64
 8   floor                 14503 non-null  int64
 9   is_apartment          1861 non-null   object
 10  studio                14503 non-null  bool
 11  open_plan             14503 non-null  bool
 12  kitchen_area          14503 non-null  float64
 13  balcony               14503 non-null  float64
 14  locality_name         14503 non-null  object
 15  airports_nearest      11539 non-null  float64
 16  cityCenters_nearest   11559 non-null  float64
 17  parks_around3000      14503 non-null  float64
 18  parks_nearest         5293 non-null   float64
 19  ponds_around3000      14503 non-null  float64
 20  ponds_nearest         5731 non-null   float64
 21  days_exposition       12050 non-null  float64
dtypes: bool(2), datetime64[ns](1), float64(14), int64(3), object(2)
memory usage: 2.9+ MB
```

Здесь выглядит логичным привести балконы, а также количество парков и водоемов к типу int

Ввод [197]:

```
columns = ['balcony', 'parks_around3000', 'ponds_around3000']
for col in columns:
    data[col] = data[col].astype(int)
```

Теперь у нас готовы данные для расчетов и работы с ними

1.2.10. Заключение

Большинство пропусков нелогичны - высота потолков, или площадь кухни. Либо сервис размещения сбоит и позволяет ввести нереальные данные, либо с выгрузкой что то не так, стоит обратиться в распределительный центр и уточнить. Но мы тут одни в поле воины, так что спасаемся тем что умеем.

Некоторые пропуски наоборот, логичные - те же самые дни размещения, или балконы. Это стоит учитывать, так как это дополнительная информация прямо в руки.

1.3. Расчёты и добавление результатов в таблицу

1.3.1. Считаем цену квадратного метра

Ввод [198]:

```
# стоимость помещения делим на его общий метраж
data['square_price'] = (data['last_price'] / data['total_area']).astype(int)
```

Ввод [199]:

```
data.head()
```

Out[199]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07	3	2.70	16.0
4	2	10000000.0	100.0	2018-06-19	2	3.03	14.0
9	18	5400000.0	61.0	2017-02-26	3	2.50	9.0
10	5	5050000.0	39.6	2017-11-16	1	2.67	12.0
13	20	3550000.0	42.8	2017-07-01	2	2.56	5.0

5 rows × 23 columns

1.3.2. Добавляем этаж помещения

Создадим удобную функцию автоматического определения этажа

Ввод [202]:

```
def floor_identify(row):
    if row['floor'] == 1:
        return 'Первый'
    if row['floor'] == row['floors_total']:
        return 'Последний'
    return 'Другой'

# применяем к датафрейму
data['floor_cat'] = data.apply(floor_identify, axis=1)

data.head()
```

Out[202]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07	3	2.70	16.0
4	2	10000000.0	100.0	2018-06-19	2	3.03	14.0
9	18	5400000.0	61.0	2017-02-26	3	2.50	9.0
10	5	5050000.0	39.6	2017-11-16	1	2.67	12.0
13	20	3550000.0	42.8	2017-07-01	2	2.56	5.0

5 rows × 24 columns

Небольшая проверка

Ввод [203]:

```
data.query('floor == 1')[['floor', 'floor_cat']].head()
```

Out[203]:

	floor	floor_cat
16	1	Первый
28	1	Первый
37	1	Первый
68	1	Первый
76	1	Первый

Ввод [204]:

```
data.query('floor == floors_total')[['floor', 'floor_cat']].head()
```

Out[204]:

	floor	floor_cat
13	5	Последний
23	9	Последний
43	4	Последний
52	2	Последний
64	5	Последний

Справились

1.3.3. Соотношения площадей

Сначала разберемся с отношением жилой к общей

Ввод [206]:

```
data['living_to_total_square'] = data['living_area'] / data['total_area']

data.head()
```

Out[206]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07	3	2.70	16.0
4	2	10000000.0	100.0	2018-06-19	2	3.03	14.0
9	18	5400000.0	61.0	2017-02-26	3	2.50	9.0
10	5	5050000.0	39.6	2017-11-16	1	2.67	12.0
13	20	3550000.0	42.8	2017-07-01	2	2.56	5.0

5 rows × 25 columns

Теперь можно узнать соотношения кухни к общей

Ввод [208]:

```
data['kitchen_to_total_square'] = data['kitchen_area'] / data['total_area']

data.head()
```

Out[208]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07	3	2.70	16.0
4	2	10000000.0	100.0	2018-06-19	2	3.03	14.0
9	18	5400000.0	61.0	2017-02-26	3	2.50	9.0
10	5	5050000.0	39.6	2017-11-16	1	2.67	12.0
13	20	3550000.0	42.8	2017-07-01	2	2.56	5.0

5 rows × 26 columns

1.3.4. День недели, месяц и год

При помощи функций пандаса создадим три новых столбца

Ввод [210]:

```
data['exposition_year'] = data['first_day_exposition'].dt.year
data['exposition_month'] = data['first_day_exposition'].dt.month
data['exposition_dayofweek'] = data['first_day_exposition'].dt.dayofweek

data.head()
```

Out[210]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07	3	2.70	16.0
4	2	10000000.0	100.0	2018-06-19	2	3.03	14.0
9	18	5400000.0	61.0	2017-02-26	3	2.50	9.0
10	5	5050000.0	39.6	2017-11-16	1	2.67	12.0
13	20	3550000.0	42.8	2017-07-01	2	2.56	5.0

5 rows × 29 columns

Теперь имеем отдельно день недели, месяц и год, с ними можно работать более удобно

1.4. Исследовательский анализ данных

1.4.1. Построение гистограмм

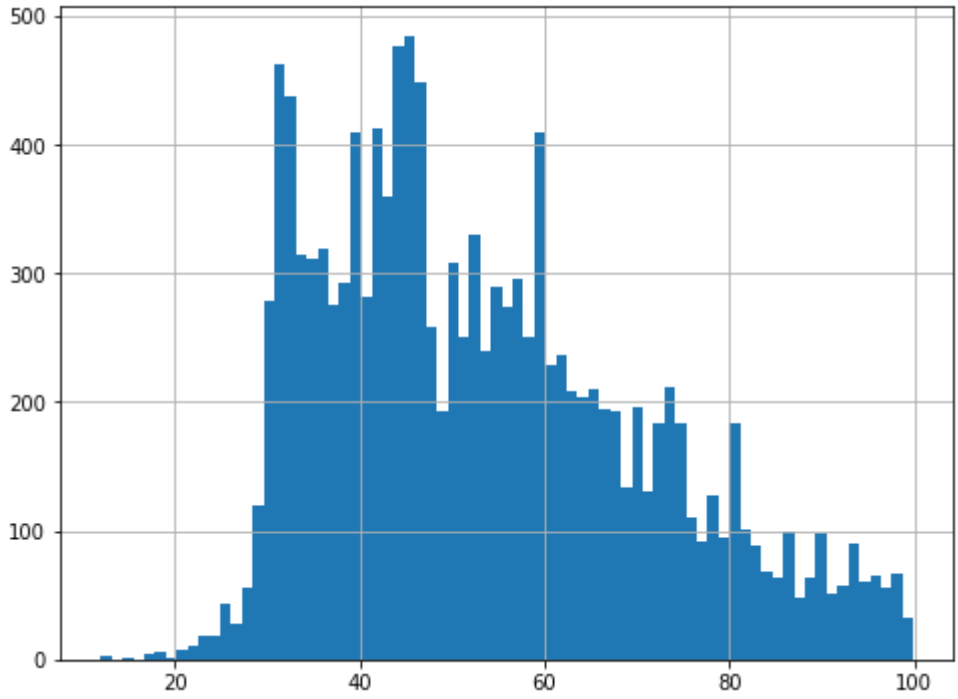
1.4.1.1. По всей площади помещения

Ввод [211]:

```
data.query('total_area < 100')['total_area'].hist(bins=75, figsize=(8, 6))
```

Out[211]:

```
<AxesSubplot:>
```



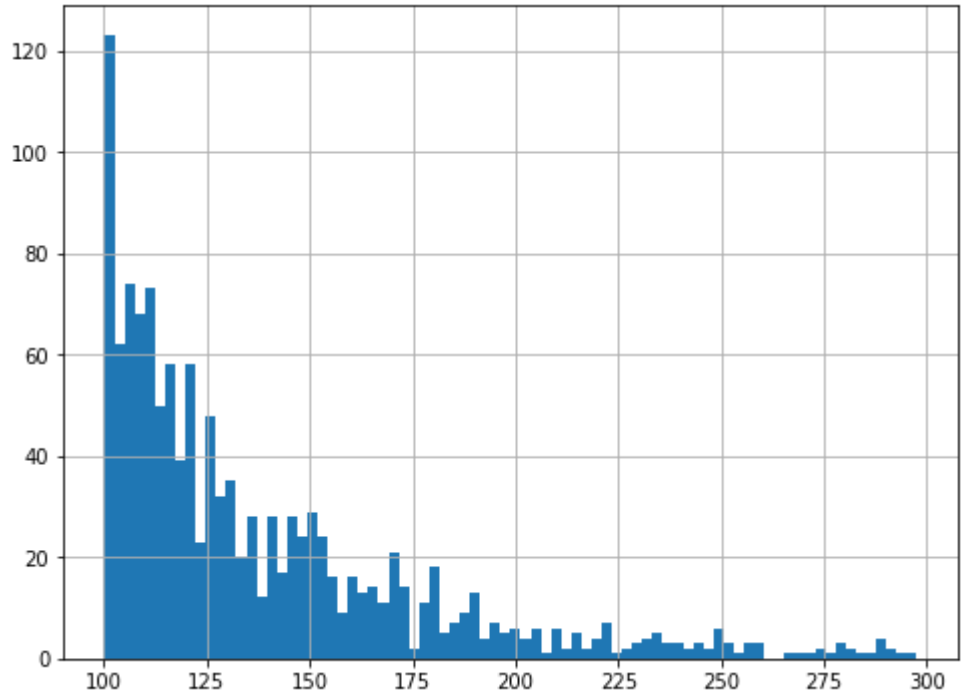
Здесь все выглядит культурно - около 30 и 45 кв. м имеются пики, но не критичные - возможно, таких квартир действительно больше

Ввод [212]:

```
data.query('total_area >= 100 and total_area < 300')['total_area'].hist(bins=80, fig
```

Out[212]:

```
<AxesSubplot:>
```



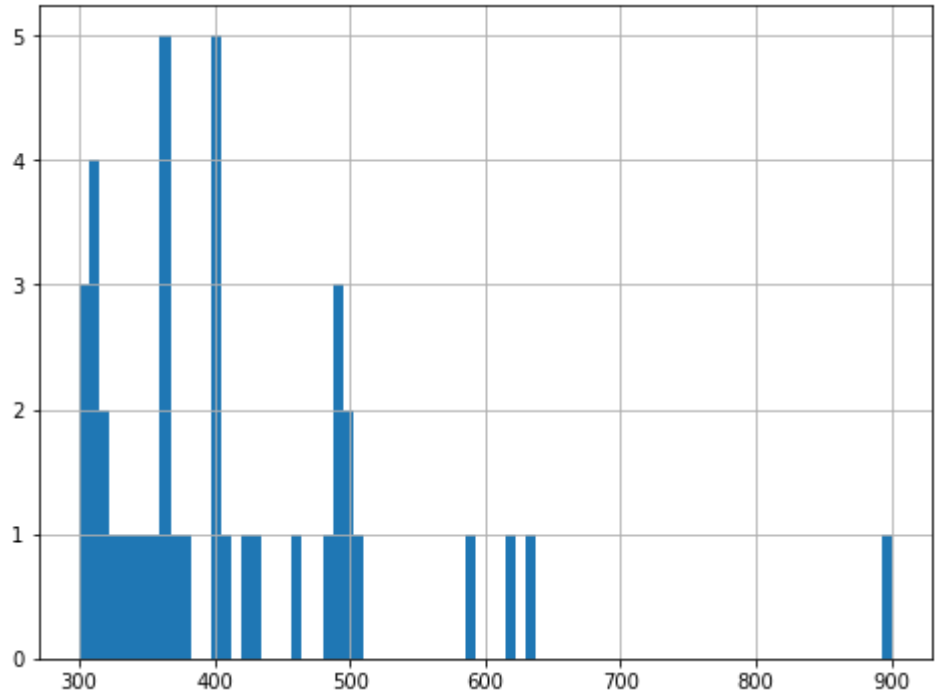
Здесь уже ситуация более странная - резкий взлет квартир по 100-102 метра

Ввод [213]:

```
data.query('total_area >= 300')['total_area'].hist(bins=80, figsize=(8, 6))
```

Out[213]:

```
<AxesSubplot:>
```



Таких квартир крайне немного, и можно увидеть, что различия у них критические - не типовые квартиры могут быть абсолютно разными

1.4.1.2. По цене продажи

Ввод [214]:

```
(data['last_price'] / 1000).describe()
```

Out[214]:

```
count     14503.000000
mean       6835.134346
std       10182.084512
min          12.190000
25%        3570.000000
50%        4850.000000
75%        7000.000000
max      420000.000000
Name: last_price, dtype: float64
```

Пройдемся по квантилям

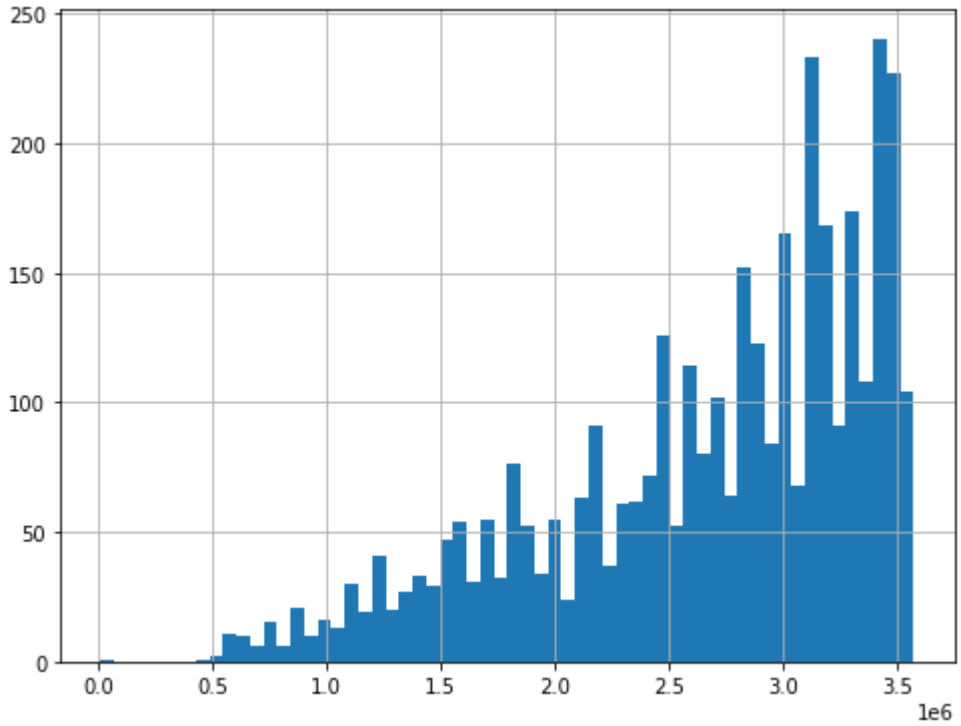
Все что дешевле 3.57 млн

Ввод [215]:

```
data.query('last_price <= 3570000')['last_price'].hist(bins=60, figsize=(8, 6))
```

Out[215]:

```
<AxesSubplot:>
```



Вполне логичный рост стандартных небольших квартир - практически линейный, с пиками в районах круглых чисел

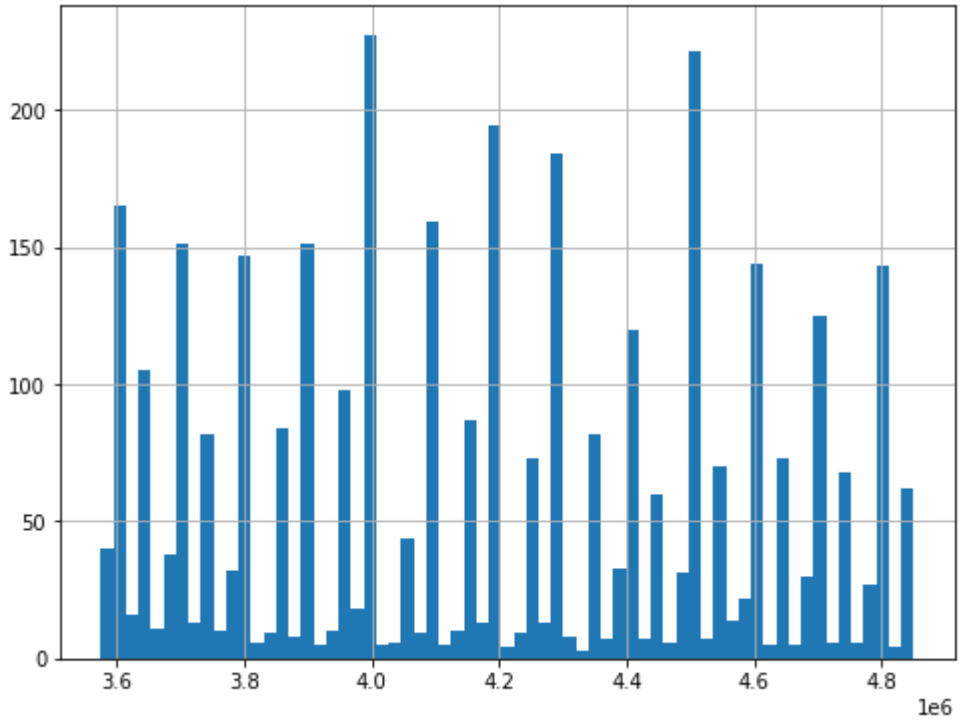
Теперь от 3.57 до 4.85 млн

Ввод [216]:

```
data.query('last_price > 3570000 and last_price <= 4850000')['last_price'].hist(bins
```

Out[216]:

```
<AxesSubplot:>
```



Уже не так линейно, однако все еще можно заметить пики именно в районах красивых чисел

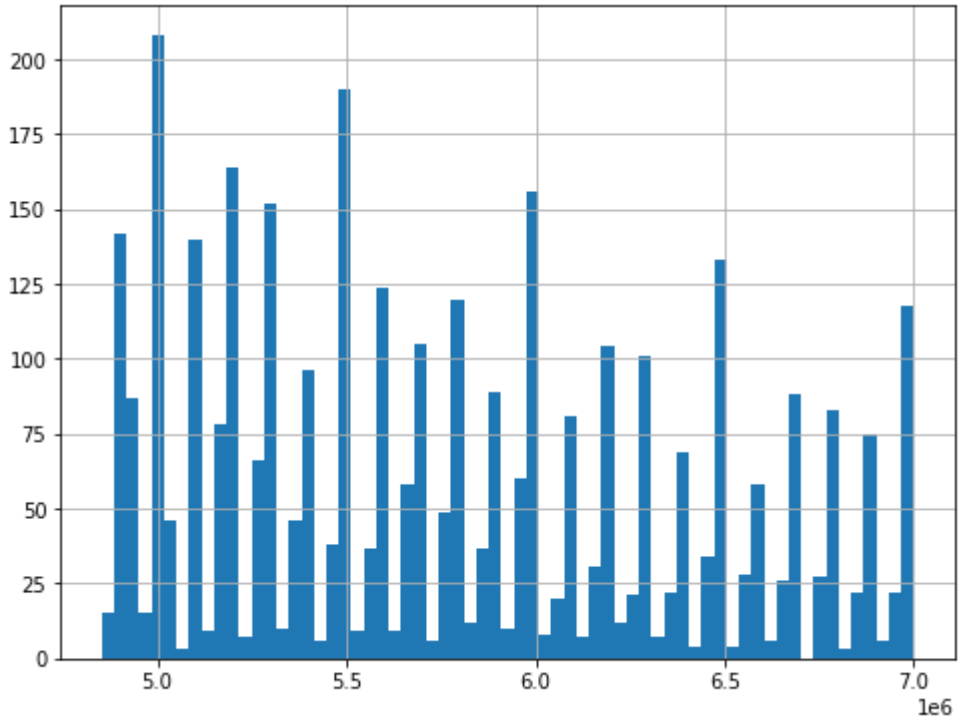
От 4.85 до 7 млн

Ввод [217]:

```
data.query('last_price > 4850000 and last_price <= 7000000')['last_price'].hist(bins
```

Out[217]:

```
<AxesSubplot:>
```



Та же самая история - чуть больше 200 квартир с приятными глазу ценами

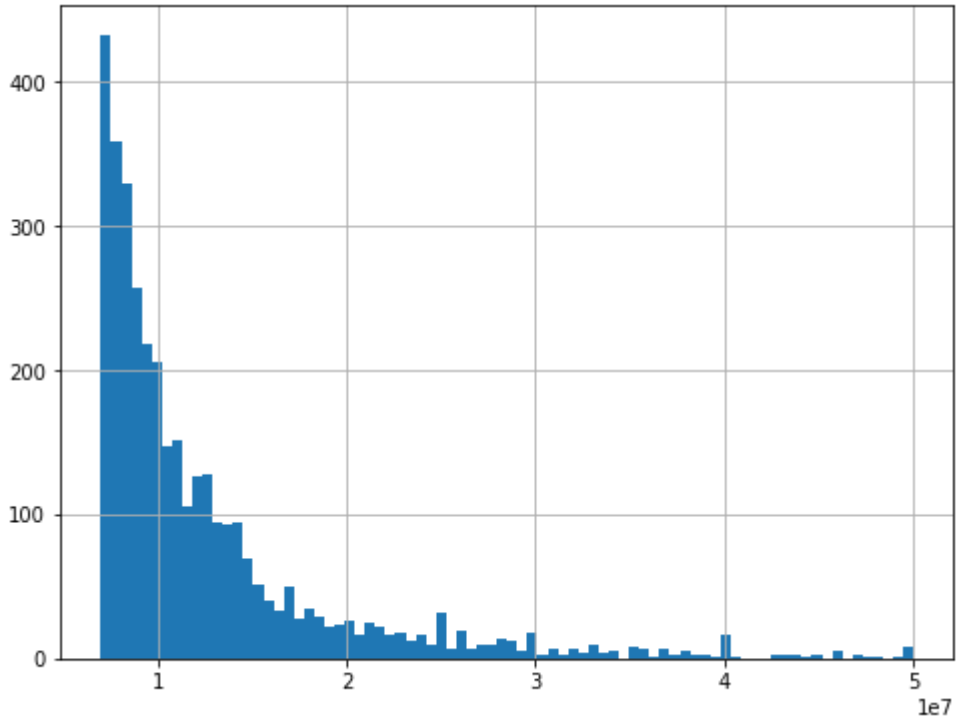
От 7 до 50 млн

Ввод [218]:

```
data.query('last_price > 7000000 and last_price <= 50000000')['last_price'].hist(bir
```

Out[218]:

```
<AxesSubplot:>
```



Тут уже тяжелая артеллерия - количество квартир с такими ценами пошло на убыль

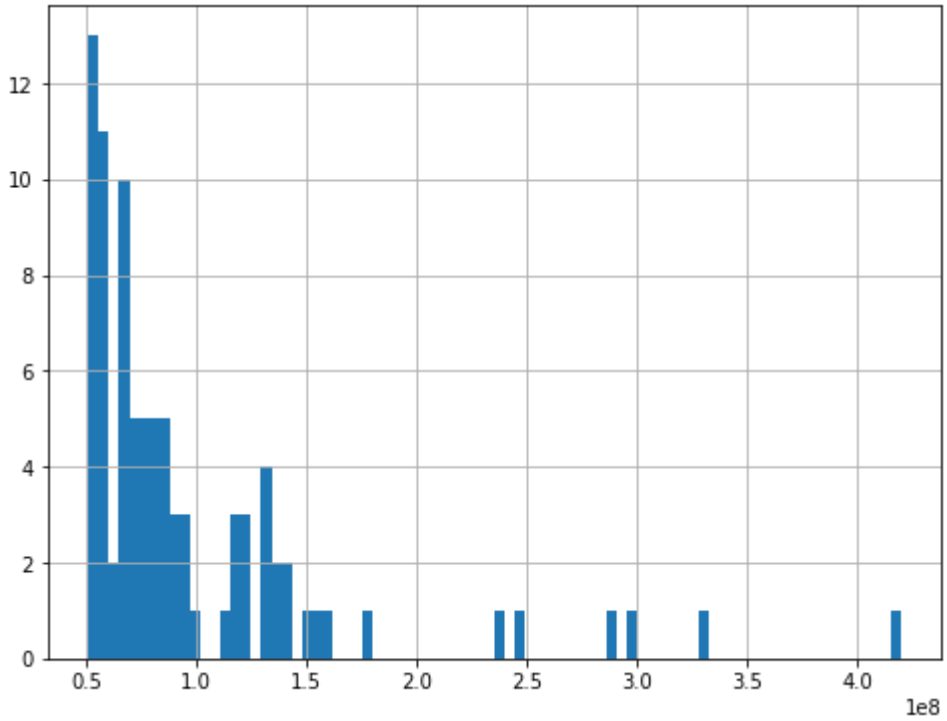
И все что больше 50 млн

Ввод [219]:

```
data.query('last_price > 50000000')['last_price'].hist(bins=80, figsize=(8, 6))
```

Out[219]:

```
<AxesSubplot:>
```



Редкие квартиры после 150 млн, редкие проблески десятков квартир между 50 и 100 млн

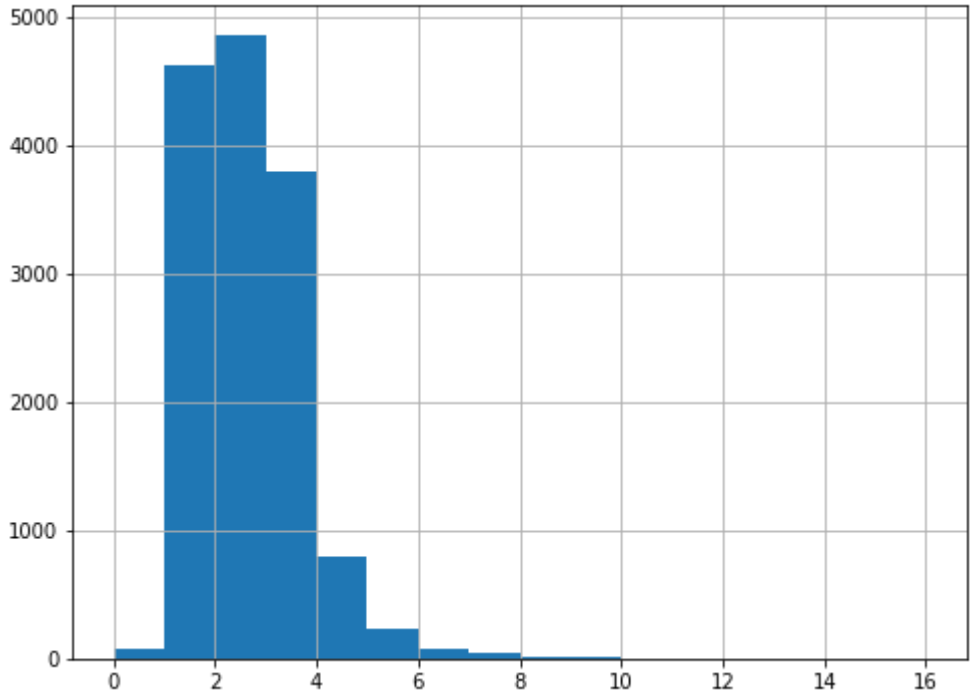
1.4.1.3. По количеству комнат

Ввод [220]:

```
data['rooms'].hist(bins=16, figsize=(8, 6))
```

Out[220]:

```
<AxesSubplot:>
```



От одной до четырех комнат - самый предпочтительный вариант, с приоритетом на две

1.4.1.4. По высоте потолков

Логика та же - делим на шаги по 1 метру и оцениваем

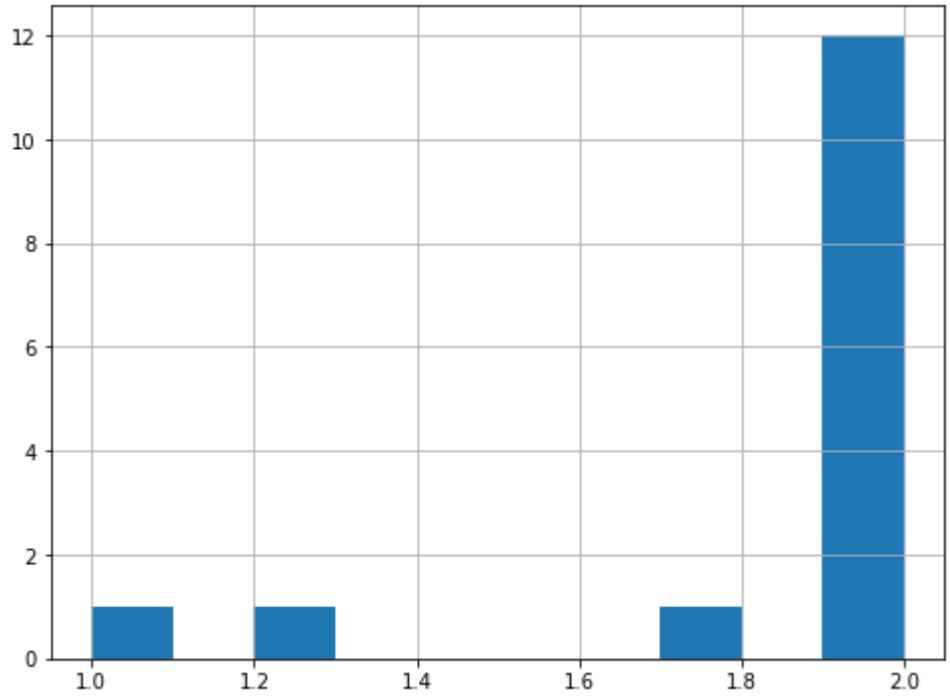
Сначала все что меньше 2 м

Ввод [221]:

```
data.query('ceiling_height <= 2')['ceiling_height'].hist(bins=10, figsize=(8, 6))
```

Out[221]:

```
<AxesSubplot:>
```



Type *Markdown* and LaTeX: α^2

По одной нестандартной квартире на 1, 1.2 и 1.7 метра высотой, и пик на 2

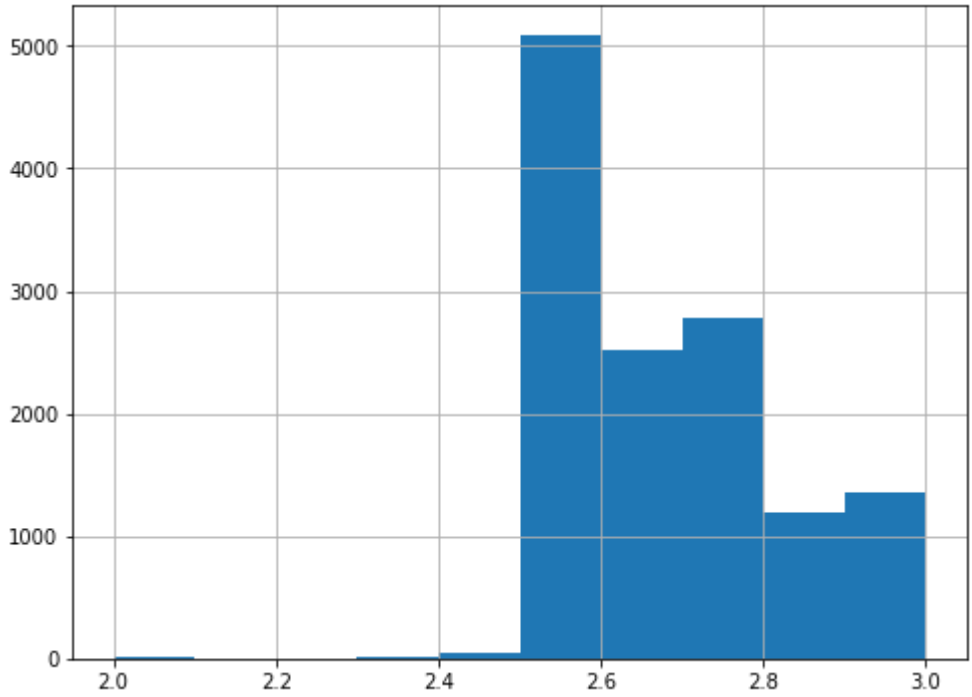
От 2 до 3 метров

Ввод [222]:

```
data.query('ceiling_height >= 2 and ceiling_height <= 3')['ceiling_height'].hist(bir
```

Out[222]:

```
<AxesSubplot:>
```



Пик на 2 метра ушел, дав место 5000+ квартир по 2.5 метра. Выглядит разумно

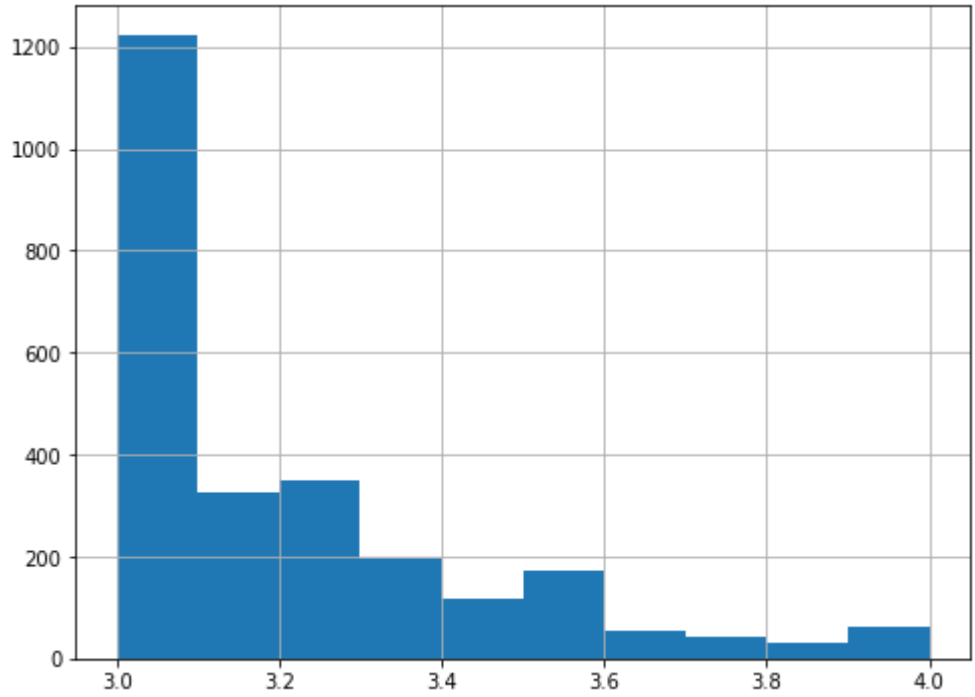
От 3 и до 4 метров

Ввод [223]:

```
data.query('ceiling_height >= 3 and ceiling_height <= 4')['ceiling_height'].hist(bir
```

Out[223]:

```
<AxesSubplot:>
```



3 метра популярны, хоть и не дотягивают до 2, а дальше спад

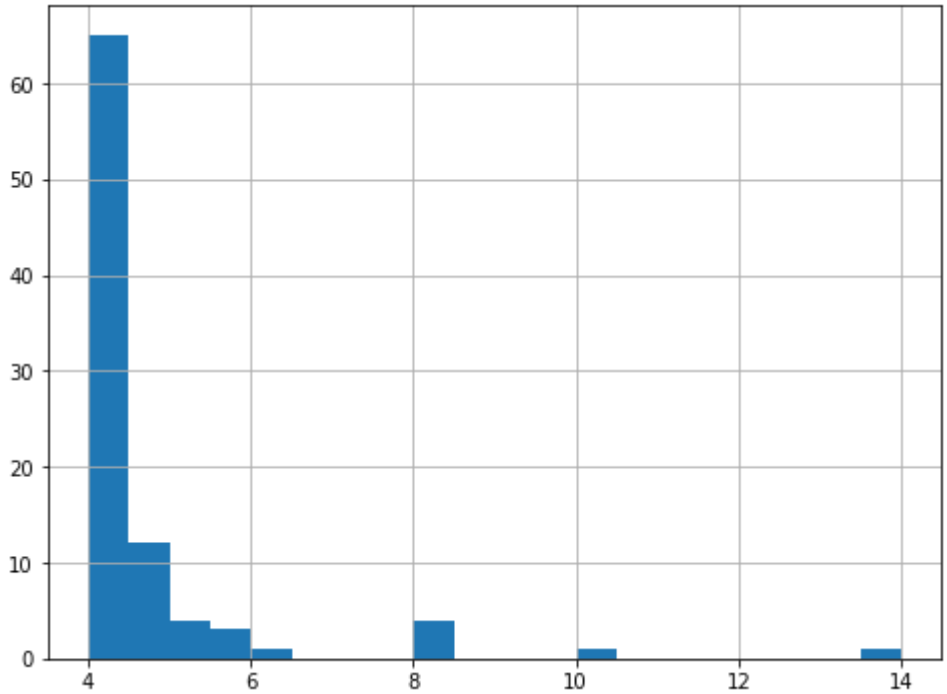
И все что от 4 и дальше

Ввод [224]:

```
data.query('ceiling_height >= 4')['ceiling_height'].hist(bins=20, figsize=(8, 6))
```

Out[224]:

```
<AxesSubplot:>
```



Под 70 с небольшим квартир с 4 метра потолками и единичные случаи большей высоты

1.4.2. Заключение

Как и в жизни, здесь влавствуют стандартные значения - большинство типовых показателей в параметрах выдают пики, цены выше +/- 3 млн ставят на круглые цифры

1.4.3. Время продажи квартиры

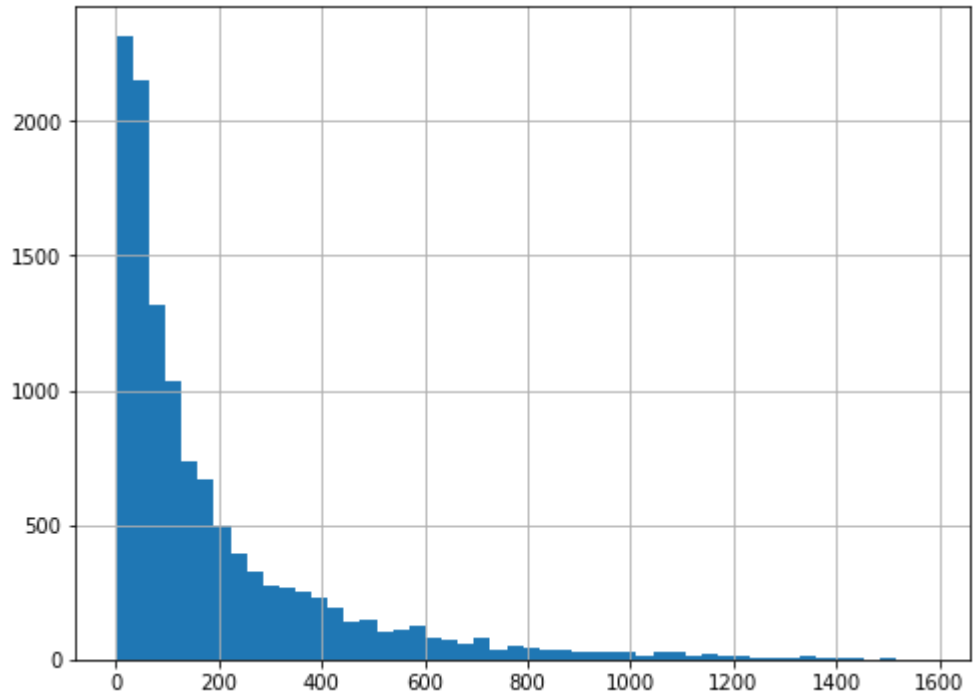
1.4.3.1. Гистограмма

Ввод [225]:

```
data['days_exposition'].hist(bins=50, figsize=(8, 6))
```

Out[225]:

```
<AxesSubplot:>
```



1.4.3.2. Редкие и пиковые значения

Уберем все что ниже 1 и выше 99 квантилей

Ввод [226]:

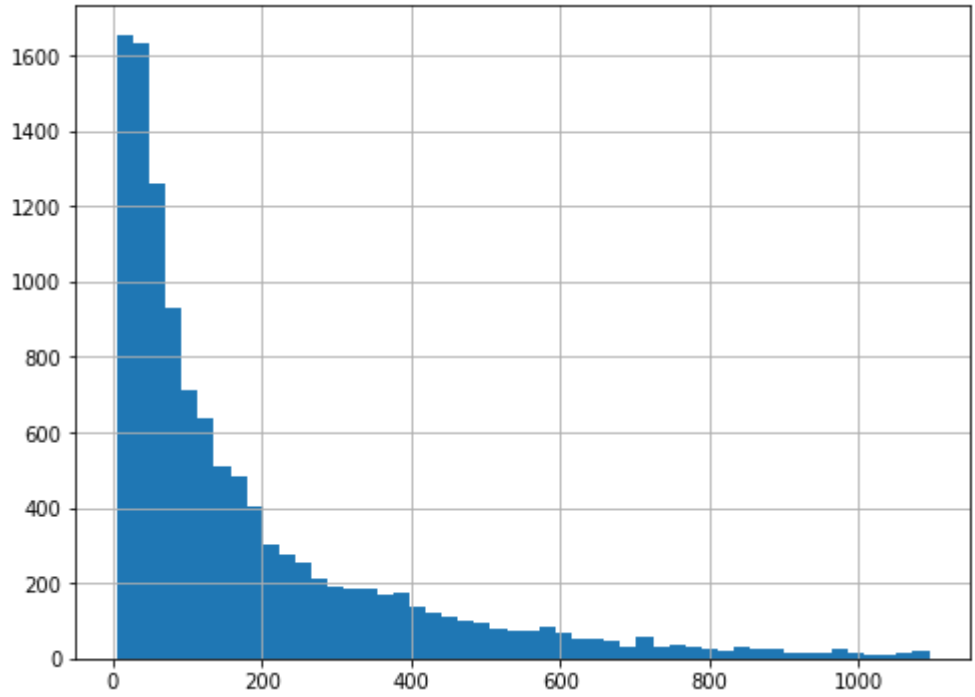
```
days_exposition_q1 = data.days_exposition.quantile(.01)
days_exposition_q99 = data.days_exposition.quantile(.99)
data['days_exposition'] = (data
                           [data.days_exposition > days_exposition_q1]
                           [data.days_exposition < days_exposition_q99]
                           ['days_exposition'])
```

Ввод [227]:

```
data['days_exposition'].hist(bins=50, figsize=(8, 6))
```

Out[227]:

```
<AxesSubplot:>
```



Получили намного меньше значений, значит, выбросов было достаточно

1.4.3.3. Среднее и медиана

Ввод [228]:

```
print('Среднее:', data['days_exposition'].mean())
print('Медиана:', data['days_exposition'].median())
```

```
Среднее: 182.70778279771545
Медиана: 104.0
```

1.4.3.4. Анализ

Среднее время продажи - 182 дня, в то время как медианное - 104 дня

По графику видно, что большую часть покупают за первые 100 дней

Самые быстрые продажи - около 10-20 дней после публикации

1.4.4. Площадь квартиры

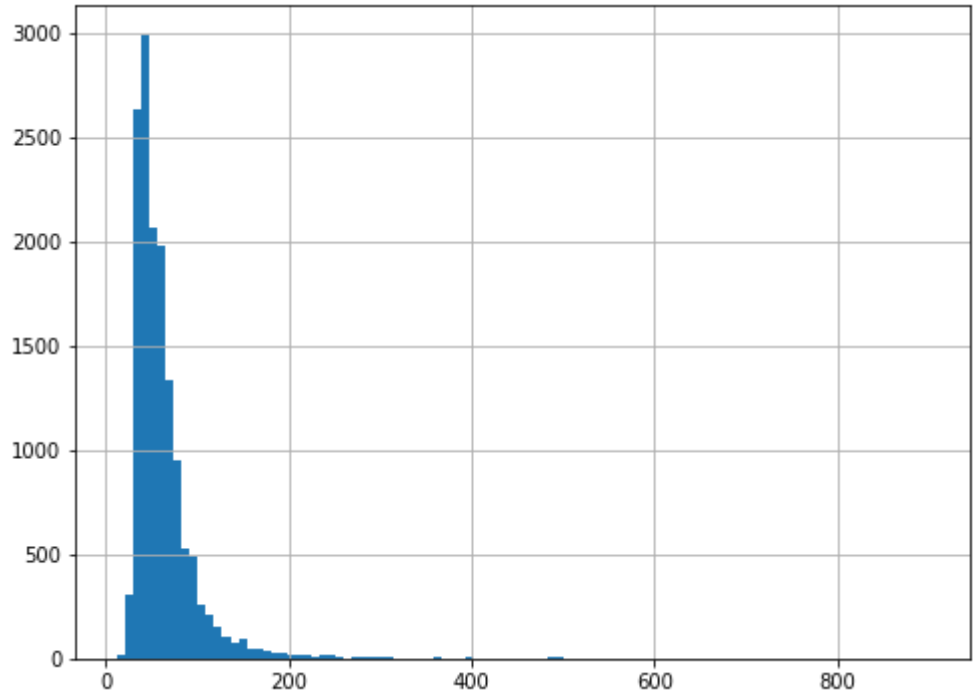
1.4.4.1. Гистограмма

Ввод [86]:

```
data['total_area'].hist(bins=100, figsize=(8, 6))
```

Out[86]:

```
<AxesSubplot:>
```



1.4.4.2. Редкие и пиковые значения

Ввод [87]:

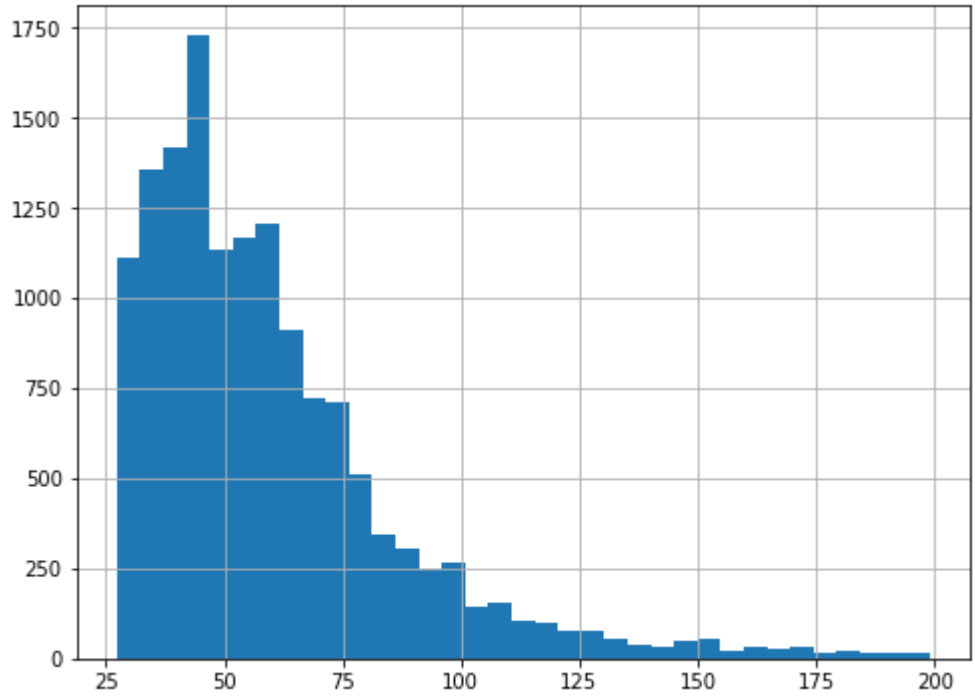
```
total_area_q1 = data.total_area.quantile(.01)
total_area_q99 = data.total_area.quantile(.99)
data['total_area'] = (data
                          [data.total_area > total_area_q1]
                          [data.total_area < total_area_q99]
                          ['total_area'])
```

Ввод [88]:

```
data['total_area'].hist(bins=35, figsize=(8, 6))
```

Out[88]:

```
<AxesSubplot:>
```



Намного более наглядно, пик около 35-40

1.4.4.3. Среднее и медиана

Ввод [89]:

```
print('Среднее:', data['total_area'].mean())
print('Медиана:', data['total_area'].median())
```

```
Среднее: 59.94330940315315
Медиана: 53.4
```

1.4.4.4. Анализ

Средняя площадь квартиры - 60 кв. м, медианная - 53 кв. м

По графику видно, что большую часть квартир составляют ~45 кв. м

Редкими квартирами можно считать те, которые больше 100 кв. м

1.4.5. Цена квартиры

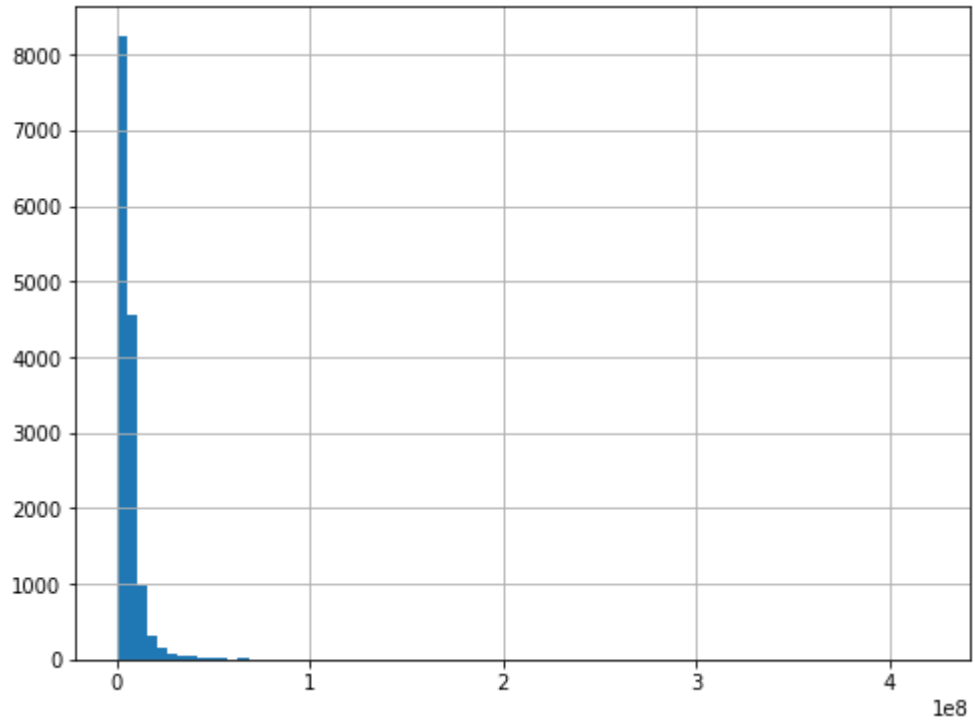
1.4.5.1. Гистограмма

Ввод [90]:

```
data['last_price'].hist(bins=80, figsize=(8, 6))
```

Out[90]:

```
<AxesSubplot:>
```



1.4.5.2. Редкие и пиковые значения

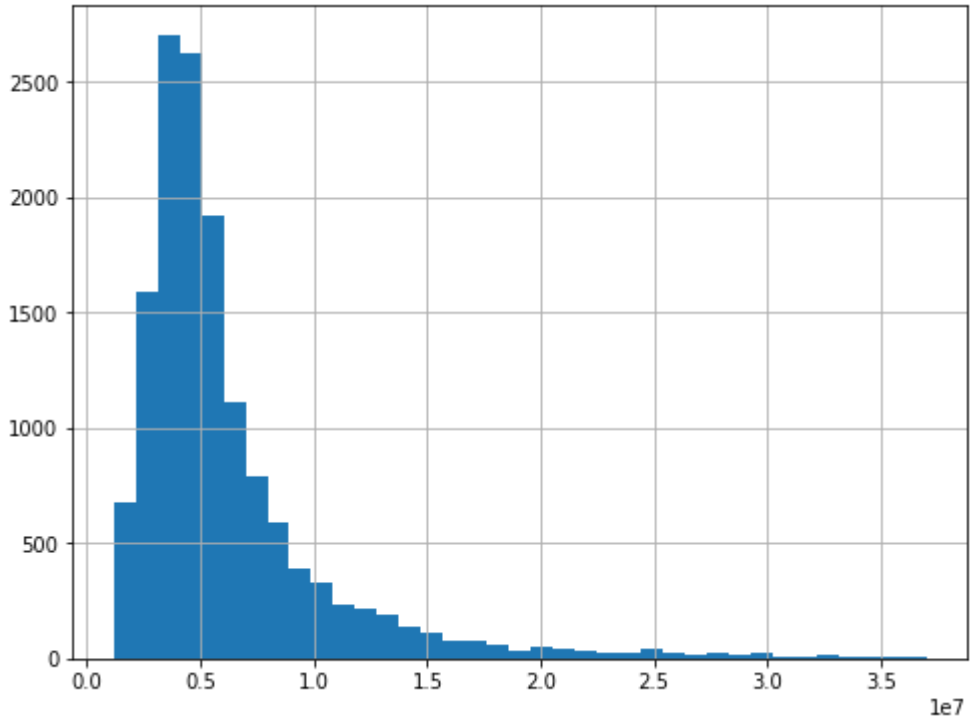
Ввод [92]:

```
last_price_q1 = data.last_price.quantile(.01)
last_price_q99 = data.last_price.quantile(.99)
data['last_price'] = (data
                            [data.last_price > last_price_q1]
                            [data.last_price < last_price_q99]
                            ['last_price'])

data['last_price'].hist(bins=37, figsize=(8, 6))
```

Out[92]:

```
<AxesSubplot:>
```



Ограничились ценой до 38 млн

1.4.5.3. Среднее и медиана

Ввод [93]:

```
print('Среднее:', data['last_price'].mean())
print('Медиана:', data['last_price'].median())
```

```
Среднее: 6181071.5278833965
Медиана: 4850000.0
```

```
Самая частая цена – от ~3 до ~6 млн

Среднее на 6.1, медиана на 4.9

Преобладают квартиры дороже 5 млн
```

1.4.6. Число комнат

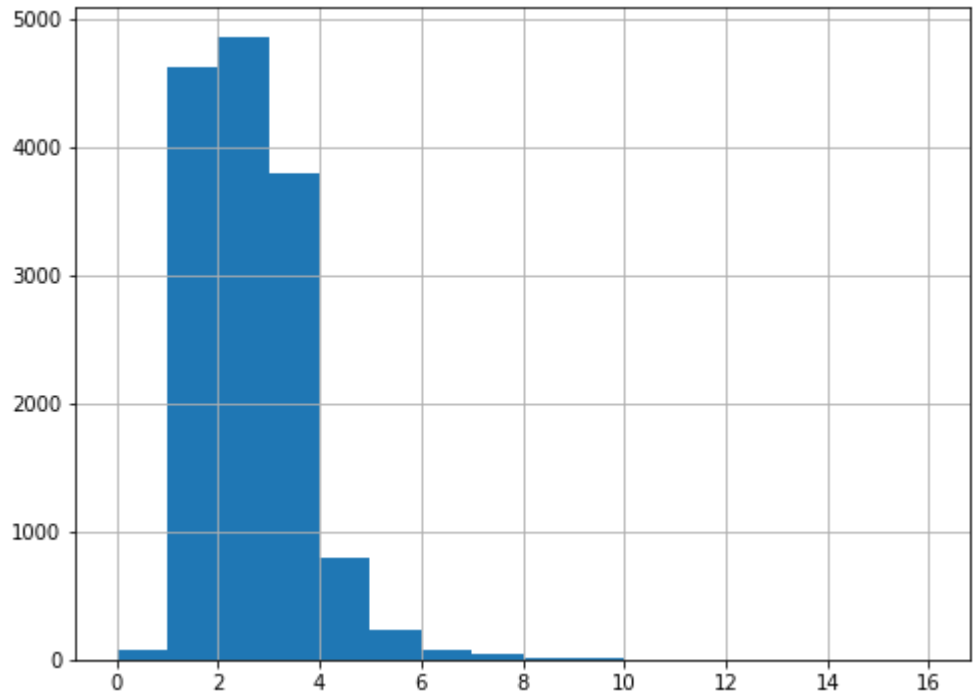
1.4.6.1. Гистограмма

Ввод [94]:

```
data['rooms'].hist(bins=16, figsize=(8, 6))
```

Out[94]:

```
<AxesSubplot:>
```



1.4.6.2. Редкие и пиковые значения

Ввод [95]:

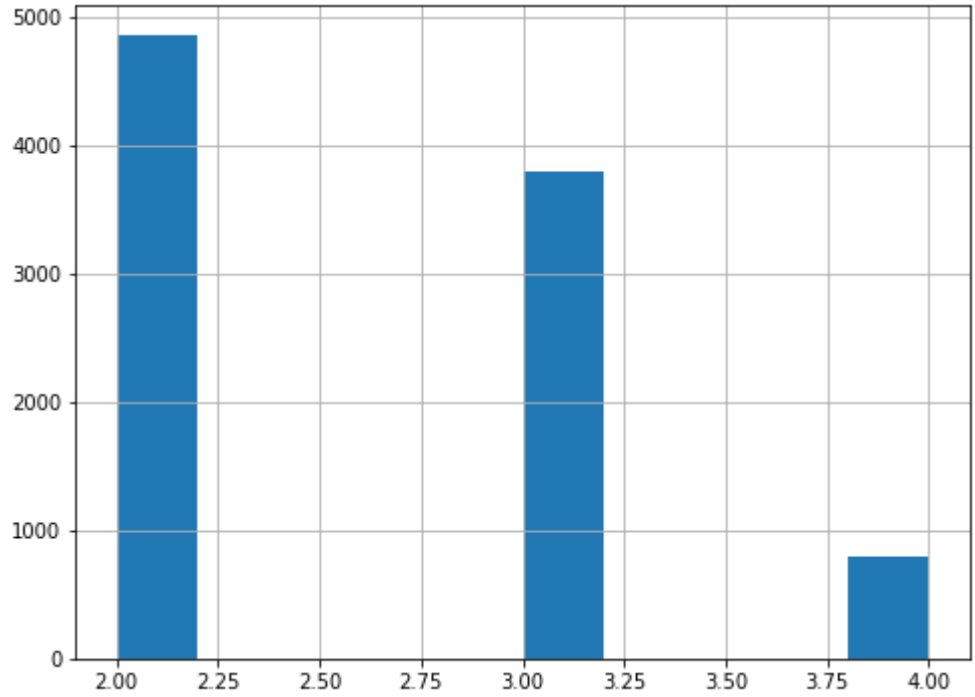
```
rooms_q1 = data.rooms.quantile(.01)
rooms_q99 = data.rooms.quantile(.99)
data['rooms'] = (data
                            [data.rooms > rooms_q1]
                            [data.rooms < rooms_q99]
                            ['rooms'])
```

Ввод [96]:

```
data['rooms'].hist(bins=10, figsize=(8, 6))
```

Out[96]:

```
<AxesSubplot:>
```



1.4.6.3. Среднее и медиана

Ввод [97]:

```
print('Среднее:', data['rooms'].mean())
print('Медиана:', data['rooms'].median())
```

```
Среднее: 2.5709291238478653
Медиана: 2.0
```

Чаще всего людей интересуют двухкомнатные квартиры, с небольшим отставанием трешки, а с четырмя комнатами - уже исключительные продажи

1.4.7. Высота потолков

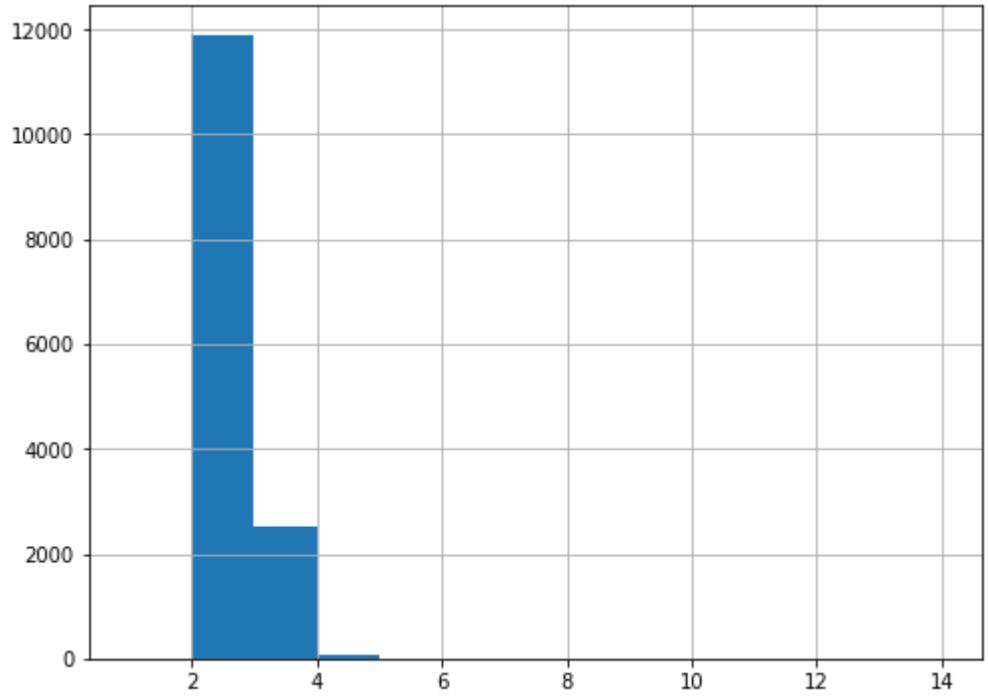
1.4.7.1. Гистограмма

Ввод [98]:

```
data['ceiling_height'].hist(bins=13, figsize=(8, 6))
```

Out[98]:

```
<AxesSubplot:>
```



1.4.7.2. Редкие и пиковые значения

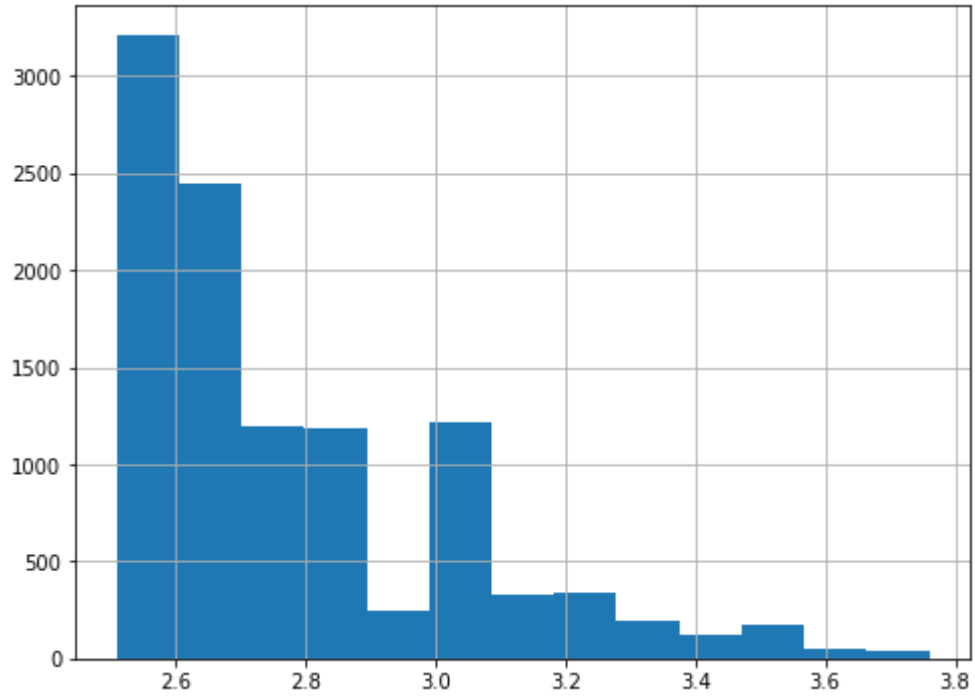
Ввод [100]:

```
ceiling_height_q1 = data.ceiling_height.quantile(.01)
ceiling_height_q99 = data.ceiling_height.quantile(.99)
data['ceiling_height'] = (data
                          [data.ceiling_height > ceiling_height_q1]
                          [data.ceiling_height < ceiling_height_q99]
                          ['ceiling_height'])

data['ceiling_height'].hist(bins=13, figsize=(8, 6))
```

Out[100]:

```
<AxesSubplot:>
```



1.4.7.3. Среднее и медиана

Ввод [101]:

```
print('Среднее:', data['ceiling_height'].mean())
print('Медиана:', data['ceiling_height'].median())
```

```
Среднее: 2.7851189812232753
Медиана: 2.7
```

Заметно, что начиная с самой популярной высоты потолка - 2.5м спрос идет по убывающей, однако, вновь появляется на стандартной высоте в 3м

1.4.8. Зависимость цены

1.4.8.1. От площади

Ввод [102]:

```
data[['last_price', 'total_area']].corr()
```

Out[102]:

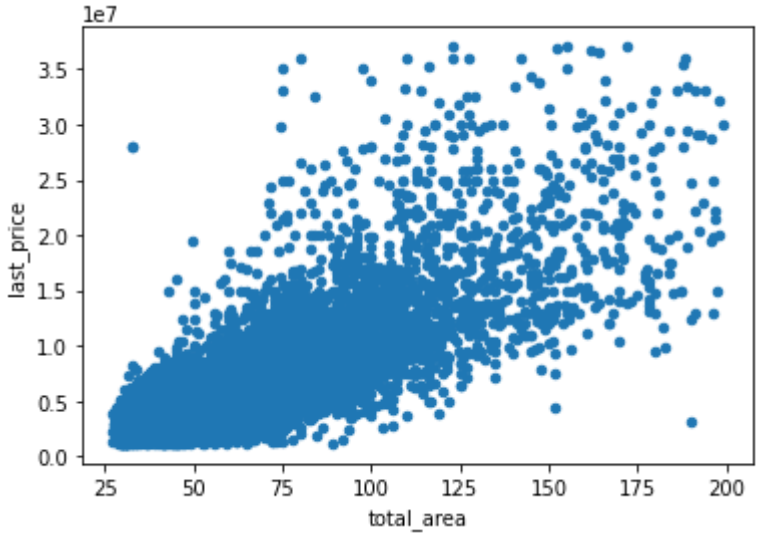
	last_price	total_area
last_price	1.00000	0.78709
total_area	0.78709	1.00000

Ввод [103]:

```
data.plot(x='total_area', y='last_price', kind='scatter')
```

Out[103]:

```
<AxesSubplot:xlabel='total_area', ylabel='last_price'>
```



Здесь видна линейная зависимость - чем больше площадь, тем выше цена

1.4.8.2. От числа комнат

Ввод [104]:

```
data[['last_price', 'rooms']].corr()
```

Out[104]:

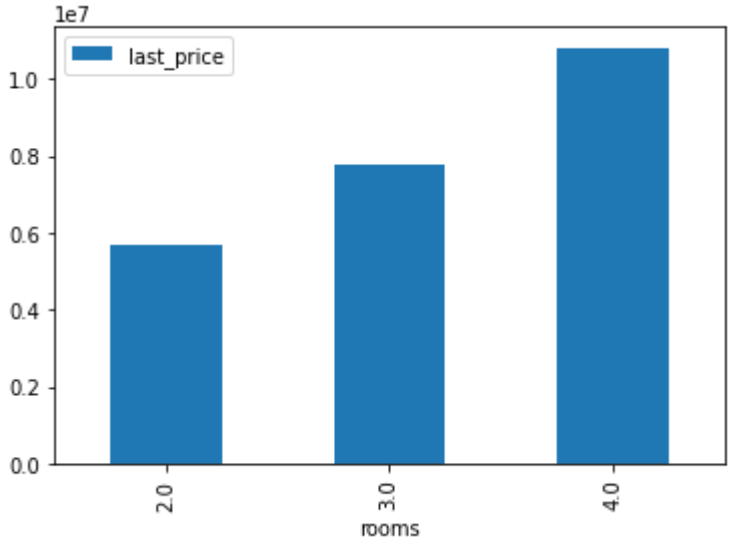
	last_price	rooms
last_price	1.000000	0.327615
rooms	0.327615	1.000000

Ввод [105]:

```
(data
 .pivot_table(index='rooms',
              values='last_price',
              aggfunc='mean')
 .plot
 .bar(y='last_price'))
```

Out[105]:

```
<AxesSubplot:xlabel='rooms'>
```



Несмотря на невысокий спрос, квартиры с четырмя комнатами, что логично, по цене выше остальных, так же заметен линейный рост стоимости

1.4.8.3. От удаленности от центра

Ввод [106]:

```
data[['last_price', 'cityCenters_nearest']].corr()
```

Out[106]:

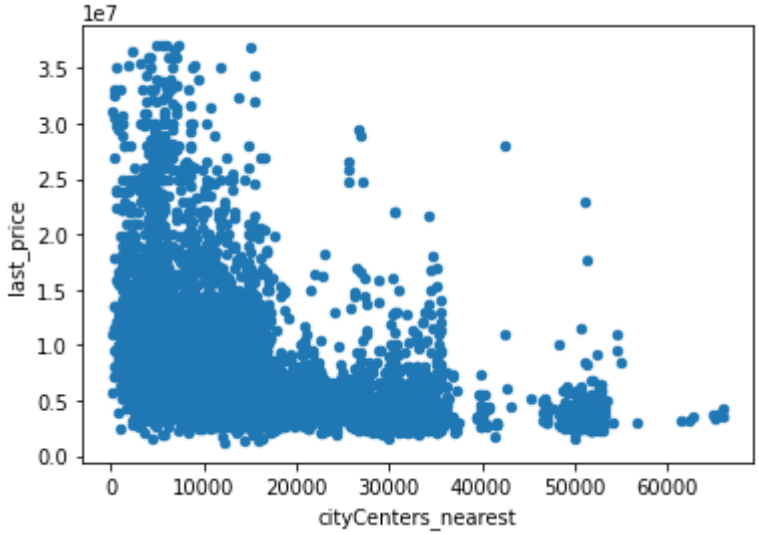
	last_price	cityCenters_nearest
last_price	1.000000	-0.342508
cityCenters_nearest	-0.342508	1.000000

Ввод [107]:

```
data.plot(x='cityCenters_nearest', y='last_price', kind='scatter')
```

Out[107]:

```
<AxesSubplot:xlabel='cityCenters_nearest', ylabel='last_price'>
```



Ввод [108]:

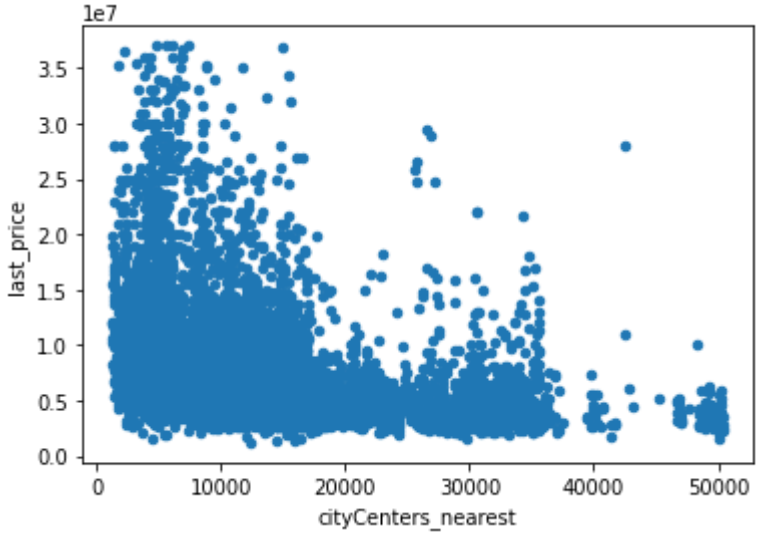
```
cityCenters_nearest_q1 = data.cityCenters_nearest.quantile(.01)
cityCenters_nearest_q99 = data.cityCenters_nearest.quantile(.99)
data['cityCenters_nearest'] = (data
                              [data.cityCenters_nearest > cityCenters_nearest_q1]
                              [data.cityCenters_nearest < cityCenters_nearest_q99]
                              ['cityCenters_nearest'])
```

Ввод [109]:

```
data.plot(x='cityCenters_nearest', y='last_price', kind='scatter')
```

Out[109]:

```
<AxesSubplot:xlabel='cityCenters_nearest', ylabel='last_price'>
```



Здесь уже можно наблюдать обратную линейную зависимость - как и разброс цены, так и максимальная стоимость выше именно в квартирах в центре, однако, есть выбросы и на более отдаленных

1.4.8.4. От этажа

Ввод [110]:

```
def floor_cat_to_numeric(row):
    if row['floor_cat'] == 'Первый':
        return 1
    if row['floor_cat'] == 'Последний':
        return 3
    return 2

data['floor_cat_numeric'] = data.apply(floor_cat_to_numeric, axis=1)

data[['last_price', 'floor_cat_numeric']].corr()
```

Out[110]:

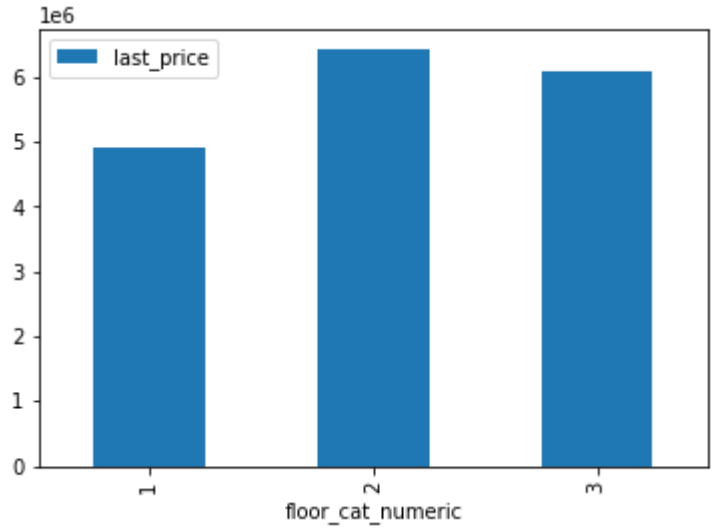
	last_price	floor_cat_numeric
last_price	1.000000	0.060511
floor_cat_numeric	0.060511	1.000000

Ввод [111]:

```
(data
 .pivot_table(index='floor_cat_numeric',
              values='last_price',
              aggfunc='mean')
 .plot
 .bar(y='last_price'))
```

Out[111]:

```
<AxesSubplot:xlabel='floor_cat_numeric'>
```



Здесь все чуть более ситуативно - в лидерах последний и между первым и последним этажи, первый же отстает, хоть и не намного

Однако, средняя цена выше именно на квартиры с промежуточными этажами

1.4.8.5. От даты размещения

Ввод [112]:

```
display(data[['last_price', 'exposition_year']].corr())
```

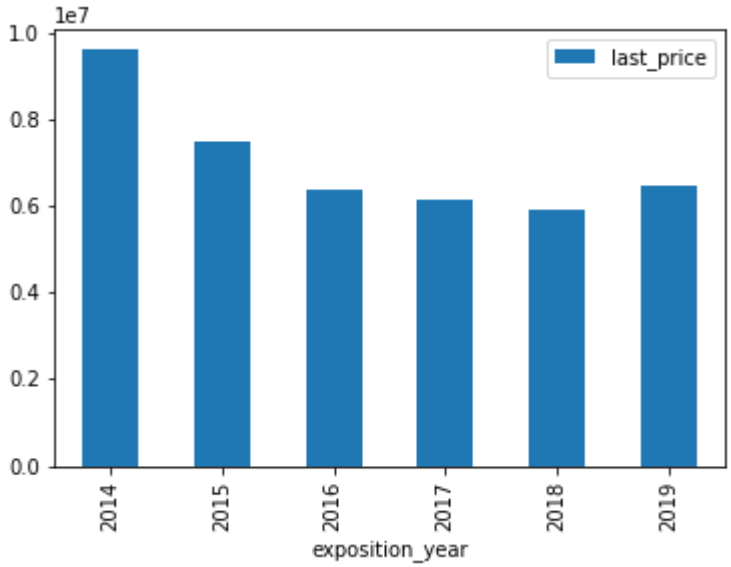
	last_price	exposition_year
last_price	1.000000	-0.045844
exposition_year	-0.045844	1.000000

Ввод [113]:

```
(data
 .pivot_table(index='exposition_year',
              values='last_price',
              aggfunc='mean')
 .plot
 .bar(y='last_price'))
```

Out[113]:

```
<AxesSubplot:xlabel='exposition_year'>
```



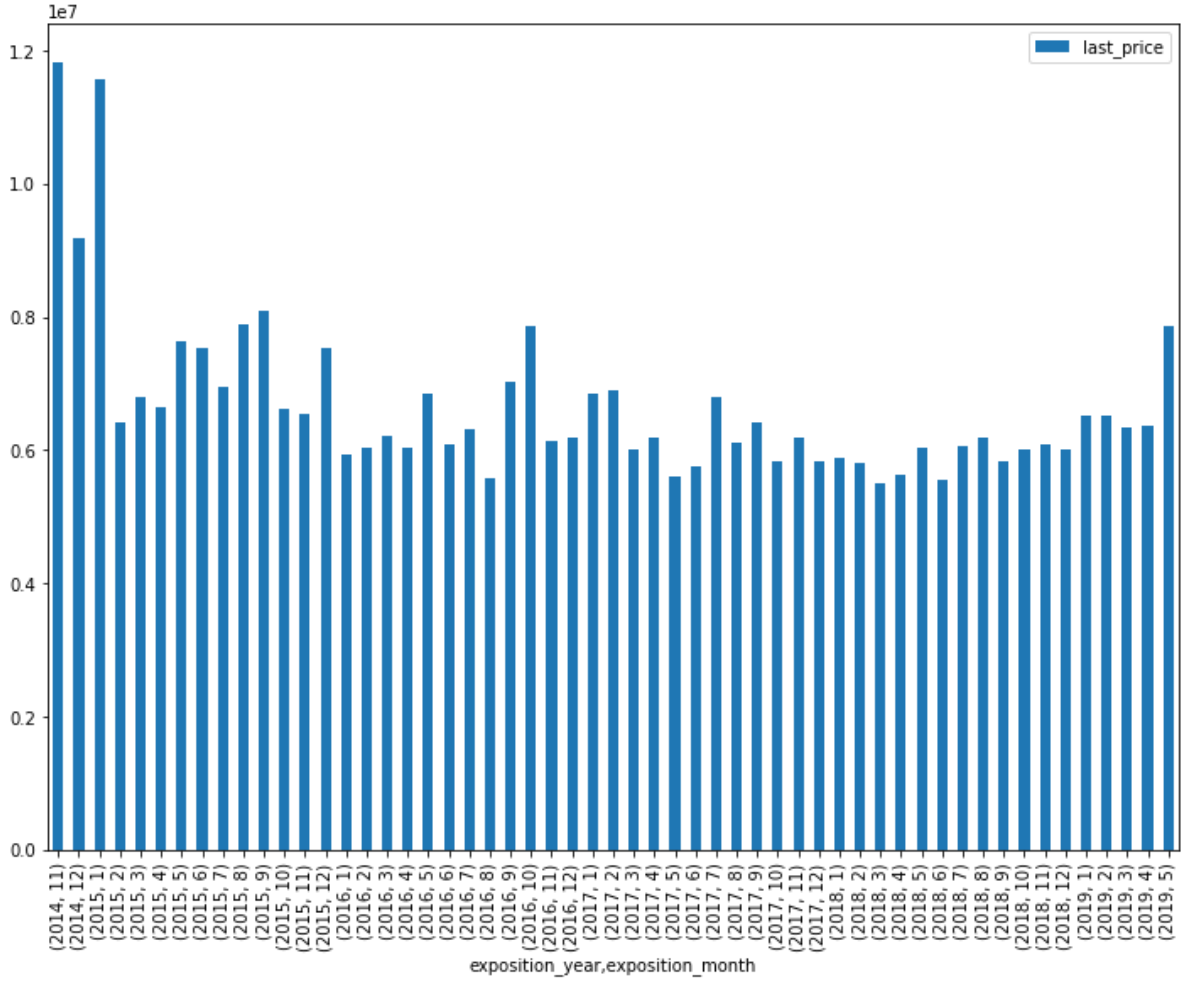
Если смотреть по годам - видна закономерность на снижение цены

Ввод [114]:

```
(data
 .pivot_table(index=['exposition_year', 'exposition_month'],
              values='last_price',
              aggfunc='mean')
 .plot
 .bar(y='last_price', figsize=(12, 9)))
```

Out[114]:

```
<AxesSubplot:xlabel='exposition_year,exposition_month'>
```



Однако, в разрезе годов и месяцев можно заметить рехкий спад в начале 2015 года, и пики спроса летом вкупе со снижением спроса ближе к зимним месяцам

1.4.8.6. Заключение

Каждый фактор влияет на стоимость квартиры, однако сильнее всех - расположение и общая площадь

Количество комнат прямо пропорционально стоимости квартиры, однако, не дает критичного роста, так же как и этаж

1.4.9. В населенных пунктах

Оставим 10 самых популярных пунктов - как завещало ТЗ

Ввод [116]:

```
locality_data = list(data
                     # составляем сводную таблицу с количеством объявлений
                     .pivot_table(index='locality_name', values='last_price', aggfur
                     # сортируем по цене в порядке убывания
                     .sort_values(by='last_price', ascending=False)
                     # берем первые 10 значений
                     .head(10)
                     # берем их названия
                     .index)


# создаем словарь для пар город - средняя цена
mean_square_price = {}
for local in locality_data:
    mean_square_price[local] = (data
                                  # снова сводная, только уже по средней цене кв. м
                                  .pivot_table(index='locality_name', values='square_
                                  # вытаскиваем эту самую цену
                                  .loc[local, 'square_price'])
```

Ввод [117]:

```
mean_square_price
```

Out[117]:

```
{'Санкт-Петербург': 115674.43412847669,
 'Всеволожск': 67344.01838235294,
 'Пушкин': 103920.02991452992,
 'посёлок Шушары': 79538.17050691244,
 'Колпино': 75938.0985221675,
 'посёлок Мурино': 87422.445,
 'Гатчина': 69992.24832214766,
 'Петергоф': 85553.67586206897,
 'посёлок Парголово': 90900.92700729927,
 'Сестрорецк': 102266.89147286821}
```

Ввод [118]:

```
# сортируем по уменьшению цены
sorted(mean_square_price, key=lambda x: mean_square_price[x], reverse=True)
```

Out[118]:

```
['Санкт-Петербург',
 'Пушкин',
 'Сестрорецк',
 'посёлок Парголово',
 'посёлок Мурино',
 'Петергоф',
 'посёлок Шушары',
 'Колпино',
 'Гатчина',
 'Всеволожск']
```

Самые дорогие квартиры - в Санкт-Петербурге

На последнем месте - Всеволожск

Ввод [119]:

```
data['mean_square_price'] = data['last_price'] / data['total_area']
```

Ввод [120]:

```
(data
 .query('not mean_square_price.isna()')
 .pivot_table(index='locality_name', values='mean_square_price', aggfunc=['count', '
 .sort_values(by=('mean', 'mean_square_price'), ascending=False))
```

Out[120]:

	count	mean
	mean_square_price	mean_square_price
locality_name		
Зеленогорск	19	113037.001954
Санкт-Петербург	9792	112060.687696
деревня Мистолово	3	107121.680644
деревня Бор	1	105539.261570
Остальные	14	105449.104044
...	...	...
поселок Торковичи	2	21383.392067
поселок городского типа Никольский	1	20862.308762
поселок Оредеж	1	19877.675841
Сланцы	7	19672.344810
Бокситогорск	1	19230.769231

259 rows × 2 columns

1.4.10. Квартиры в Санкт-Петербурге

Выделим только квартиры в Санкт-Петербурге

Ввод [121]:

```
spb_data = data.query('locality_name == "Санкт-Петербург" and not cityCenters_neares
```

Приведем расстояние к типу int и поделим на 1000 - это и будет округлением до целых километров

Ввод [122]:

```
spb_data['integer_nearest_center'] = (spb_data['cityCenters_nearest'] // 1000).astyp
```

Ввод [123]:

```
spb_data.head()
```

Out[123]:

	total_images	last_price	total_area	first_day_exposition	rooms	ceiling_height	floors_total
0	20	13000000.0	108.0	2019-03-07	3.0	2.70	16.0
4	2	10000000.0	100.0	2018-06-19	2.0	3.03	14.0
9	18	5400000.0	61.0	2017-02-26	3.0	NaN	9.0
10	5	5050000.0	39.6	2017-11-16	NaN	2.67	12.0
16	11	6700000.0	82.0	2017-11-23	3.0	3.05	5.0

5 rows × 32 columns

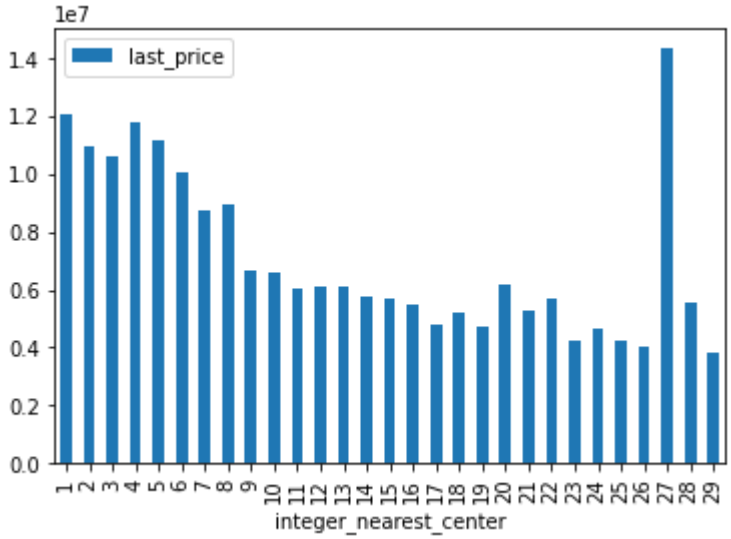
Посмотрим на график цены / удаленности

Ввод [124]:

```
spb_data.pivot_table(index='integer_nearest_center', values='last_price', aggfunc='m
```

Out[124]:

```
<AxesSubplot:xlabel='integer_nearest_center'>
```



Граница сильного изменения показателей графика - около 5 км

Там и будет центр

1.4.11. Квартиры в центре Санкт-Петербурга

Оставим только центровские квартиры

Ввод [125]:

```
spb_center_data = spb_data.query('integer_nearest_center <= 5')
```

Посмотрим на описание данных в них

Ввод [126]:

```
spb_center_data[['total_area', 'last_price', 'rooms', 'ceiling_height']].describe()
```

Out[126]:

	total_area	last_price	rooms	ceiling_height
count	1641.000000	1.663000e+03	1295.000000	1593.000000
mean	87.522657	1.130240e+07	2.841699	3.094212
std	36.517510	6.347436e+06	0.748284	0.267053
min	28.600000	1.600000e+06	2.000000	2.510000
25%	60.200000	6.990000e+06	2.000000	2.900000
50%	80.500000	9.300000e+06	3.000000	3.050000
75%	108.600000	1.350000e+07	3.000000	3.300000
max	197.900000	3.700000e+07	4.000000	3.760000

В общей площади среднее 87, медиана - 80, значит, половина квартир стоит значительно выше, что завышает среднее

Цена продажи выглядит так же - среднее 1.1, медиана - 0.9

Комнаты и высота потолков на одном уровне медианы и среднего

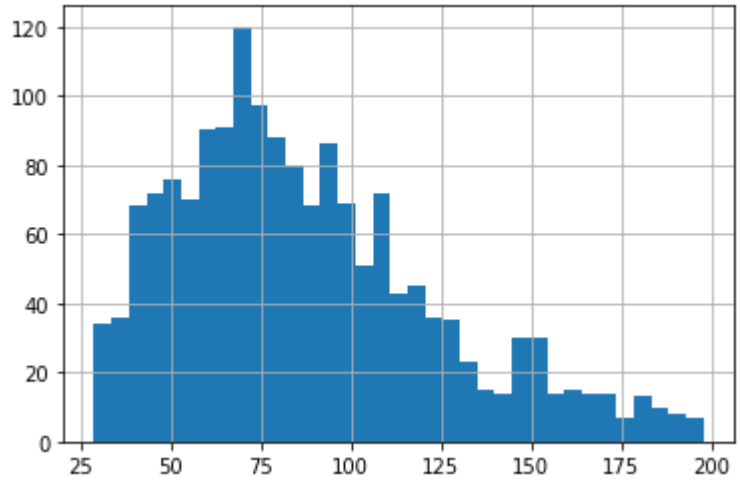
Посмотрим на гистограмму площади квартир в центре

Ввод [127]:

```
spb_center_data['total_area'].hist(bins=35)
```

Out[127]:

```
<AxesSubplot:>
```



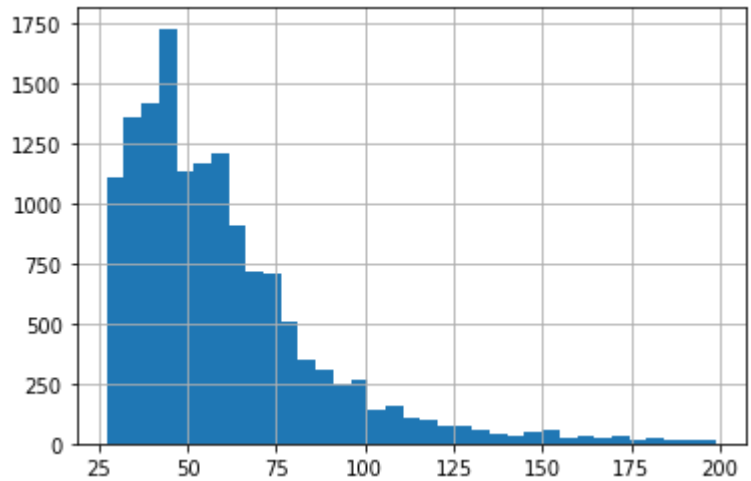
И в общих данных

Ввод [128]:

```
data['total_area'].hist(bins=35)
```

Out[128]:

```
<AxesSubplot:>
```



Заметно, что большинство квартир из общих данных перетягивают площадь в меньшую сторону, у них с 75 кв.м начинается спад, в то время как это же значение является пиковым для квартир в центре

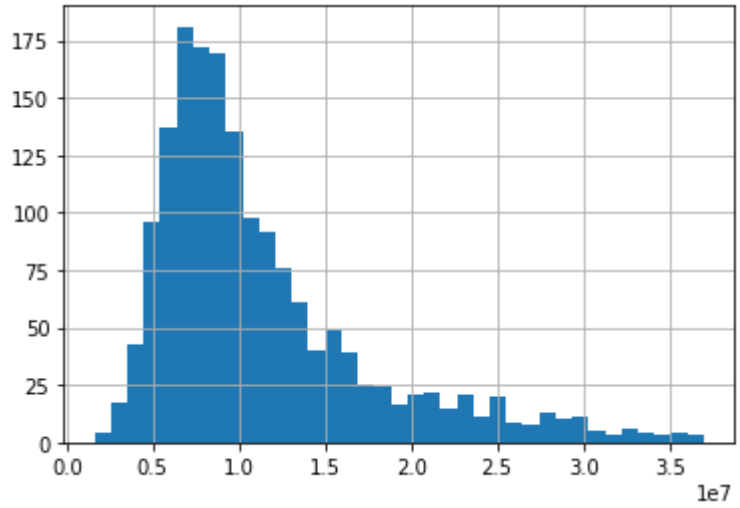
Оценим так же по цене

Ввод [129]:

```
spb_center_data['last_price'].hist(bins=37)
```

Out[129]:

```
<AxesSubplot:>
```

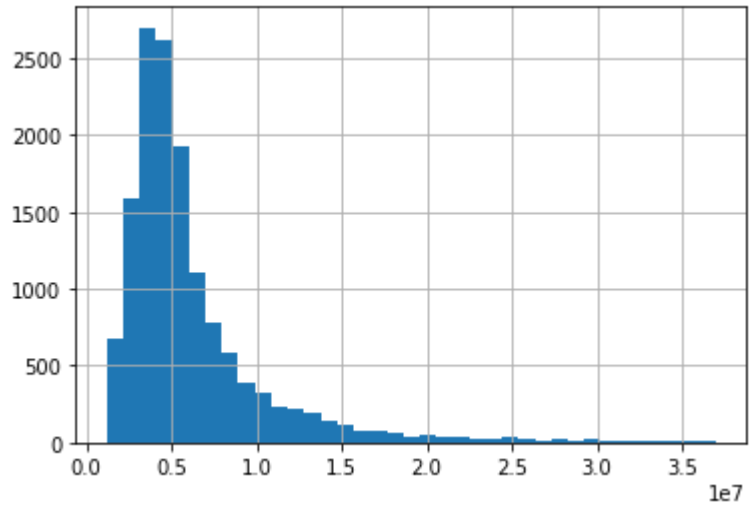


Ввод [130]:

```
data['last_price'].hist(bins=37)
```

Out[130]:

```
<AxesSubplot:>
```



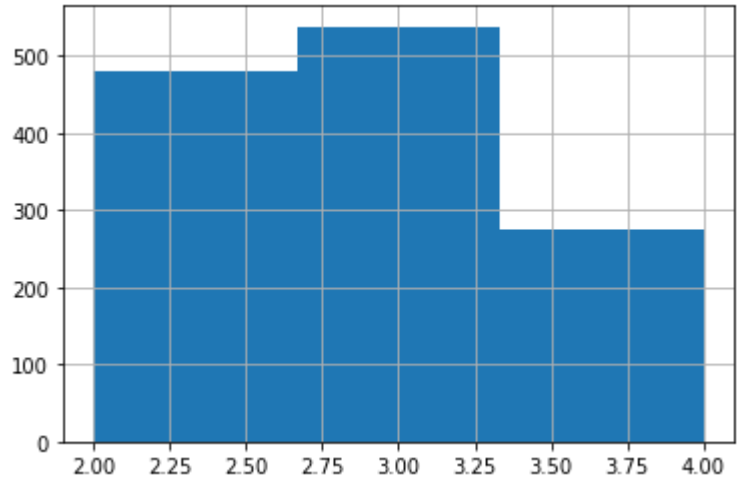
Картина очень похожа - пики общих цен выходят на 3-4 млн, а в центре - на 6-8 млн

Ввод [131]:

```
spb_center_data['rooms'].hist(bins=3)
```

Out[131]:

```
<AxesSubplot:>
```

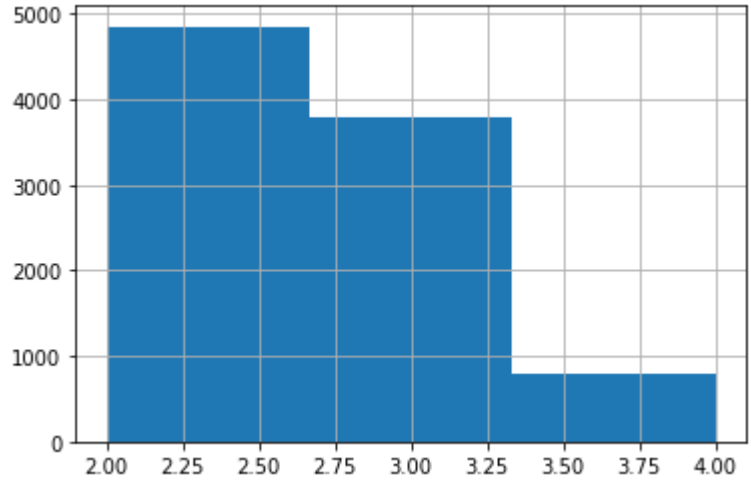


Ввод [132]:

```
data['rooms'].hist(bins=3)
```

Out[132]:

```
<AxesSubplot:>
```



Здесь критически меняются местами только 2 и 3 комнаты - в центре предпочитают тройные, в общей массе - с двумя

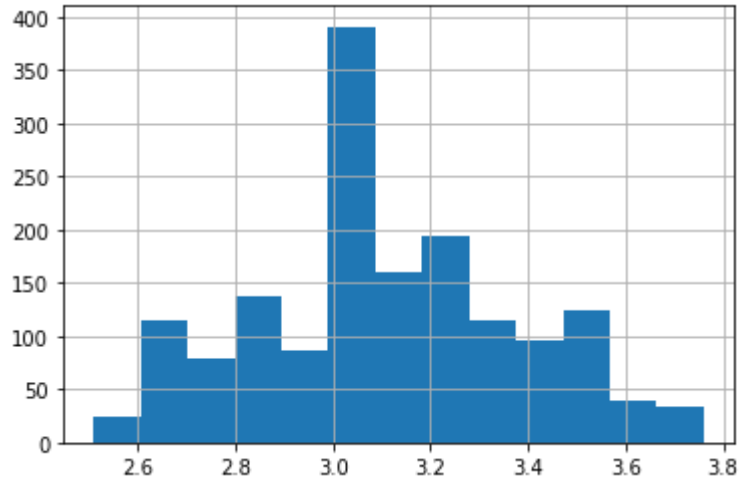
Исследуем высоту потолков

Ввод [133]:

```
spb_center_data['ceiling_height'].hist(bins=13)
```

Out[133]:

```
<AxesSubplot:>
```

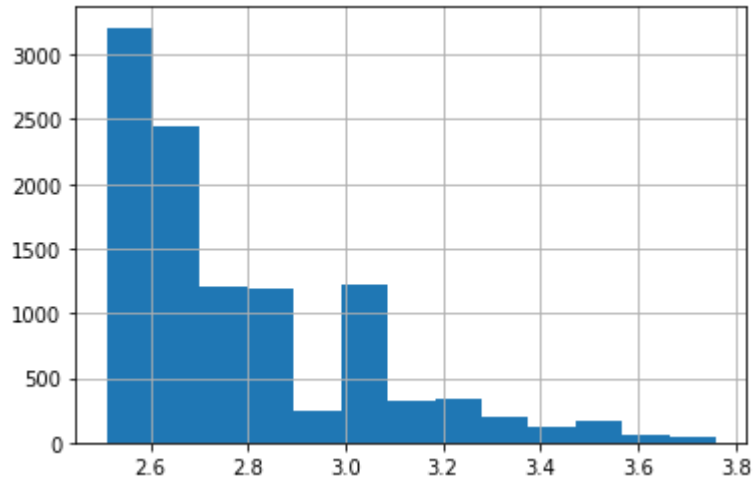


Ввод [134]:

```
data['ceiling_height'].hist(bins=13)
```

Out[134]:

```
<AxesSubplot:>
```



Здесь ситуация более интересная - три метра над уровнем пола все еще остаются заметным пиком, но в отличии от (я так подозреваю, ~старых~ исторических) квартир в центре, общая масса новых застроек стремится к значением 2.4-2.5 м

Построим зависимость цены от различных показателей

Ввод [135]:

```
# от количества комнат
spb_center_data[['last_price', 'rooms']].corr()
```

Out[135]:

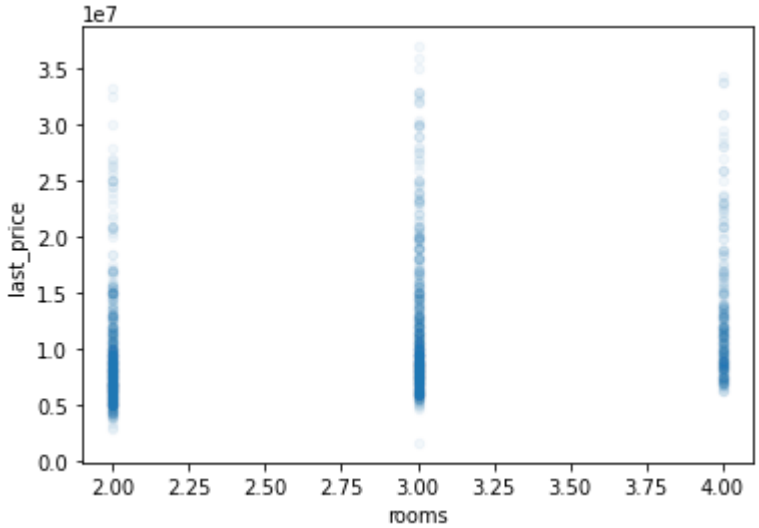
	last_price	rooms
last_price	1.000000	0.221977
rooms	0.221977	1.000000

Ввод [136]:

```
spb_center_data.plot(y='last_price', x='rooms', kind='scatter', alpha=0.05)
```

Out[136]:

```
<AxesSubplot:xlabel='rooms', ylabel='last_price'>
```



Цена драматически не меняется, но виден рост при увеличении комнат

Ввод [137]:

```
# от этажа помещения
spb_center_data[['last_price', 'floor']].corr()
```

Out[137]:

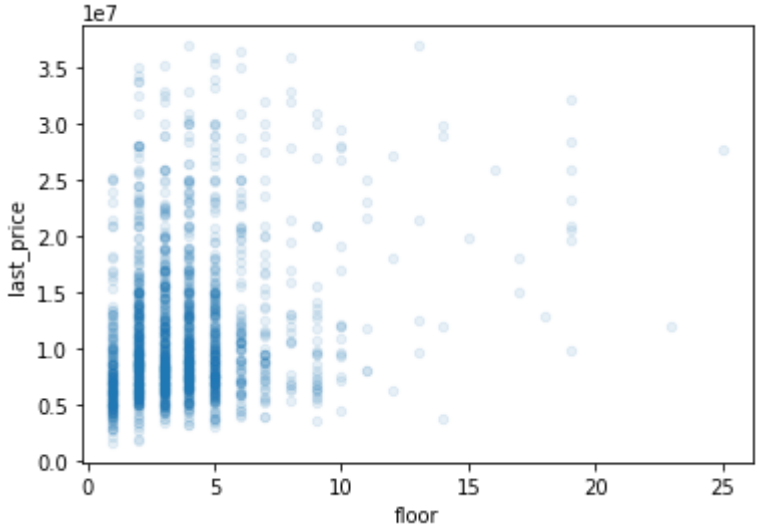
	last_price	floor
last_price	1.000000	0.237353
floor	0.237353	1.000000

Ввод [138]:

```
spb_center_data.plot(y='last_price', x='floor', kind='scatter', alpha=0.1)
```

Out[138]:

```
<AxesSubplot:xlabel='floor', ylabel='last_price'>
```



Здесь заметно, что на первых ~ этажах цены не сильно отличаются друг от друга, однако, начиная с 15 этажа видно увеличение общей стоимости квартиры, даже с учетом меньшего количества объявлений

Ввод [139]:

```
# от удаленности от центра
spb_center_data[['last_price', 'integer_nearest_center']].corr()
```

Out[139]:

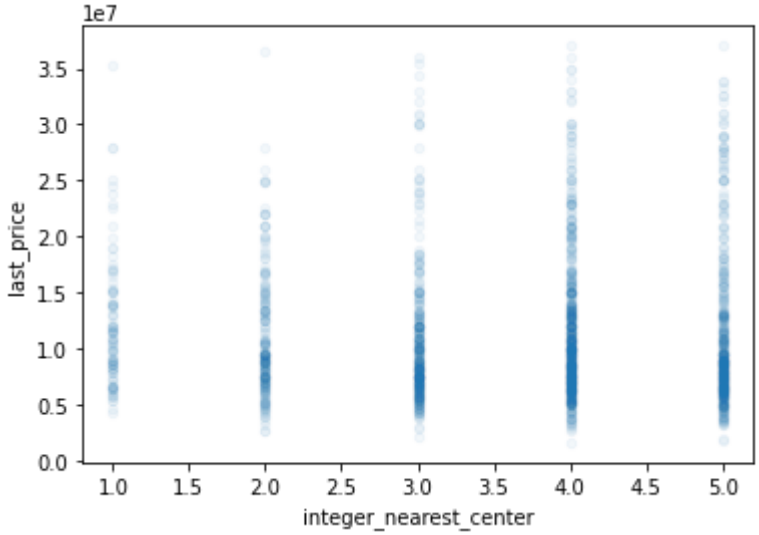
	last_price	integer_nearest_center
last_price	1.000000	0.005505
integer_nearest_center	0.005505	1.000000

Ввод [140]:

```
spb_center_data.plot(y='last_price', x='integer_nearest_center', kind='scatter', alp
```

Out[140]:

```
<AxesSubplot:xlabel='integer_nearest_center', ylabel='last_price'>
```



Здесь так же - предложений больше, чем дальше от центра, однако, цены выше именно в центре

Ввод [141]:

```
# от дня размещения объявления
spb_center_data[['last_price', 'exposition_dayofweek']].corr()
```

Out[141]:

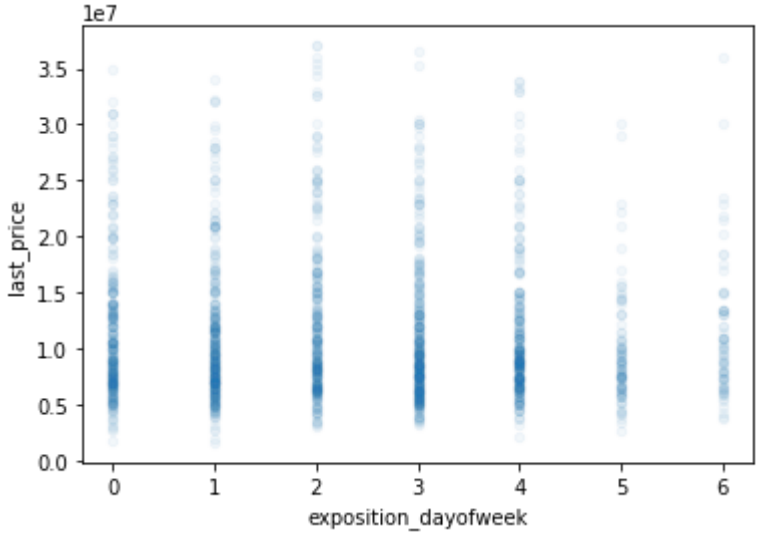
	last_price	exposition_dayofweek
last_price	1.000000	-0.038563
exposition_dayofweek	-0.038563	1.000000

Ввод [142]:

```
spb_center_data.plot(y='last_price', x='exposition_dayofweek', kind='scatter', alpha
```

Out[142]:

```
<AxesSubplot:xlabel='exposition_dayofweek', ylabel='last_price'>
```



Здесь видно лишь то, что выходные значительно понижают цену

1.5. Общий вывод

Получено: датасет с данными квартир и информацией об их продаже

Исправлено: заменены отсутствующие значения там, где это возможно и критично: высота потолков, размеры самой квартиры и т.д.

Так же заменены отсутствующие значения там, где это логично, как, например, с балконами

Отделены друг от друга расстояния (переведены в км), дни, месяцы и годы продажи

Основные выводы:

1. На цену сильно вляет местоположение квартиры - квартиры в центре продаются по гораздо большей цене

2. За центр считаем все на расстоянии в 5 км

3. Исключительные квартиры создают большие выбросы - после чистки квартилей

ниже 1 и выше 99 данные заметно преображаются

4. Количество комнат слабо влияет на цену квартиры, особенно не в центре

5. Высота потолков для центра – 3 метра для подавляющей части квартир, и 2.5 метра для квартир в регионах

6. Среднее время продажи квартиры – до 80-100 дней, самые частые быстрые продажи – в первые 2-3 недели

7. Цена сильно меняется при нахождении квартиры выше 15 этажа